

```
package com.greenline.yihuantong.common.baseclass;
import android.os.Bundle;
import com.greenline.yihuantong.common.InjectHelper;
import com.greenline.yihuantong.util.AppTrackHelper;
import com.weddoctor.guahao.devkit.DevKitFragment;
public class BaseFragment extends DevKitFragment {
    protected AppTrackHelper mAppTrackHelper;
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        if (isInjectAppTrackHelper()) {
            mAppTrackHelper = new AppTrackHelper(getActivity(), this.getClass().getName(),
injectRequestCount());
        }
    }
    protected int injectRequestCount() {
        return 0;
    }
    protected boolean isInjectAppTrackHelper() {
        return false;
    }
    protected void trackRequestSuccess(String requestName) {
        if (mAppTrackHelper != null) {
            mAppTrackHelper.sendBroadcast(requestName);
        }
    }
    protected void trackRequestFailed() {
        if (mAppTrackHelper != null) {
            mAppTrackHelper.onFailed();
        }
    }
    @Override
    public void onDestroy() {
        super.onDestroy();
        if (mAppTrackHelper != null) {
            mAppTrackHelper = null;
        }
    }
    protected <O extends Object> O bind(Class<O> cls) {
        return InjectHelper.inject(cls);
    }
}

package com.greenline.yihuantong.common.baseclass;
import android.app.Activity;
import android.database.DataSetObserver;
import android.view.LayoutInflater;
import android.widget.BaseAdapter;
import java.util.List;
public abstract class BaseItemListAdapter<E> extends BaseAdapter {
    protected Activity context;
```

```
protected List<E> items;
protected LayoutInflater inflater;
public BaseItemListAdapter(Activity context, List<E> list) {
    this.context = context;
    inflater = LayoutInflater.from(context);
    this.items = list;
}
@Override
public int getCount() {
    return items.size();
}
@Override
public Object getItem(int position) {
    return items.get(position);
}
@Override
public long getItemId(int position) {
    return position;
}
public List<E> getItems() {
    return items;
}
public void setItems(List<E> e) {
    this.items = e;
    notifyDataSetChanged();
}

public void setItem(int index, E item) {
    items.set(index, item);
}

@Override
public void unregisterDataSetObserver(DataSetObserver observer) {
    if (observer != null) {
        super.unregisterDataSetObserver(observer);
    }
}

public void updateData(List<E> e) {
    this.items = e;
    notifyDataSetChanged();
}
public void addData(E item) {
    items.add(item);
    notifyDataSetChanged();
}

public void addLocation() {
    notifyDataSetChanged();
}
```

```

}
package com.greenline.yihuantong.common.baseclass;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.view.animation.AnimationUtils;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.HeaderViewListAdapter;
import android.widget.ListView;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;
import androidx.fragment.app.Fragment;
import androidx.loader.app.LoaderManager;
import androidx.loader.content.Loader;
import com.greenline.yihuantong.R;
import com.greenline.yihuantong.util.ExceptionUtils;
import com.greenline.yihuantong.util.ViewUtils;
import com.guahao.android.utils.WYToastUtils;
import java.util.ArrayList;
import java.util.List;
public abstract class BaseItemListFragment<E> extends Fragment implements
LoaderManager.LoaderCallbacks<List<E>> {
    protected List<E> items = new ArrayList<E>();
    protected ListView listView;
    protected TextView emptyView;
    protected ProgressBar progressBar;
    protected boolean listShown;
    private RefreshListener mRefreshListener;
    @Override
    public void onActivityCreated(Bundle savedInstanceState) {
        super.onActivityCreated(savedInstanceState);
        if (!items.isEmpty()) {
            setListShown(true, false);
        }
    }
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {
        return inflater.inflate(R.layout.gh_base_item_list, container, false);
    }
    @Override
    public void onDestroyView() {
        listShown = false;
        emptyView = null;
        progressBar = null;
        listView = null;
        super.onDestroyView();
    }
}

```

```

@Override
public void onViewCreated(View view, Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);
    listView = (ListView) view.findViewById(android.R.id.list);
    listView.setOnItemClickListener(new OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
            onItemClick((ListView) parent, view, position, id);
        }
    });
    progressBar = (ProgressBar) view.findViewById(R.id.pb_loading);
    emptyView = (TextView) view.findViewById(android.R.id.empty);
    configureList(getListView());
    LoaderManager.getInstance(this).initLoader(0, null, this);
}

protected void configureList(ListView listView) {
    listView.setAdapter(createAdapter(items));
}

@Override
public void onLoadFinished(Loader<List<E>> loader, List<E> items) {
    Exception exception = getException(loader);
    if (exception != null) {
        setEmptyText(ExceptionUtils.formatExceptionMessage(exception));
    } else {
        setEmptyText(getNoDataIndication());
        this.items = items;
        getListAdapter().setItems(this.items);
    }
    showList();
    if (mRefreshListener != null) {
        mRefreshListener.onRefreshComplete();
    }
    LoaderManager.getInstance(this).destroyLoader(0);
}

    .set("IsDeleted", 1);
    mapper.update(new BoxOrder(), updateWrapper);
}

/**
 *
 * @param boxOrderDetailList
 * @return
 */
private List<BoxOrdeCertify> genBoxOrdeCertify(List<BoxOrderDetail> boxOrderDetailList) {
    List<BoxOrdeCertify> boxOrdeCertifyList = new ArrayList<>();
    for (BoxOrderDetail boxOrderDetail : boxOrderDetailList) {
        BoxOrdeCertify boxOrdeCertify = new BoxOrdeCertify();
        boxOrdeCertify.setBoxOrderId(boxOrderDetail.getOrderId());
        boxOrdeCertify.setBoxOrderNo(boxOrderDetail.getOrderNo());
        boxOrdeCertify.setBoxOrderDetailId(boxOrderDetail.getId());
        boxOrdeCertify.setBookingNo(boxOrderDetail.getBookingNo());
    }
}

```

```

        boxOrdeCertify.setSupplierId(boxOrderDetail.getSupplierId());
        boxOrdeCertify.setSupplierNo(boxOrderDetail.getSupplierNo());
        boxOrdeCertify.setSupplierName(boxOrderDetail.getSupplierName());
        boxOrdeCertify.setBoxType(boxOrderDetail.getBoxType());
        boxOrdeCertify.setQuantity(boxOrderDetail.getQuantity());
        boxOrdeCertify.setUnit(boxOrderDetail.getUnit());
        boxOrdeCertify.setBookingDetailId(boxOrderDetail.getBookingDetailId());
        boxOrdeCertifyList.add(boxOrdeCertify);
    }
    return boxOrdeCertifyList;
}

/**
 *
 * @param boxOrderId
 */
@Override
public void confirmBoxOrder(String boxOrderId) {
    UpdateWrapper<BoxOrder> updateWrapper = new UpdateWrapper<BoxOrder>()
        .eq("Id", boxOrderId)
        .set("OrderStatus", 1);
    int update = mapper.update(new BoxOrder(), updateWrapper);
    if (update != 1) throw new UpdateException(", ");
}

/**
 *
 * @param isReplaceId
 * @return
 */
@Override
public List<BoxOrder> findBoxOrderList(String isReplaceId) {
    String userId = StringUtils.isNotBlank(isReplaceId) ? isReplaceId : TokenUtil.getUserId();
    return mapper.findBoxOrderList(userId);
}

/**
 * ID
 * @param orderId
 * @return
 */
@Override
public BoxOrder findBoxOrder(String bookingDetailId) {
    BoxOrder boxOrder = mapper.findBoxOrder(bookingDetailId);
    if (boxOrder != null) {
        List<FileUpload> fileUploadList = fileUploadService.getPaperList(bookingDetailId, "BOX");
        boxOrder.setPaperList(fileUploadList);
        UserInfo storeKeeper = orderService.getStoreKeeper(boxOrder.getSupplierId());
        String driver = bookingDetailMapper.selectById(bookingDetailId).getBookingUserName();
        UserInfo monitor = userInfoService.findMonitor();
        boxOrder.setStoreKeeper(storeKeeper.getName());
        boxOrder.setDriver(driver);
        boxOrder.setMonitor(monitor.getName());
    }
}

```

```
    }
    return boxOrder;
}
/**
 *
 * @return
 */
@Override
public List<OutOrder> getBoxOrderExamineList() {
    return orderMapper.getOrderExamineList("BoxOrder", TokenUtil.getUserId(), TokenUtil.getRoleCode());
}
/**
 * BookingId
 * @param bookingId
 * @return
 */
@Override
public List<BoxOrder> findBoxOrderListByBookingId(String bookingId) {
    return mapper.findBoxOrderListByBookingId(TokenUtil.getUserId(), bookingId);
}
}

package com.xinzhou.svcs.mobile.emp.service.impl;
import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.conditions.update.UpdateWrapper;
import com.xinzhou.svcs.common.ex.InsertException;
import com.xinzhou.svcs.common.ex.ServiceException;
import com.xinzhou.svcs.common.ex.UpdateException;
import com.xinzhou.svcs.common.utils.DateUtil;
import com.xinzhou.svcs.common.utils.TokenUtil;
import com.xinzhou.svcs.common.utils.UploadUtil;
import com.xinzhou.svcs.common.utils.WxUtil;
import com.xinzhou.svcs.mobile.driver.entity.Booking;
import com.xinzhou.svcs.mobile.driver.entity.BookingDetail;
import com.xinzhou.svcs.mobile.driver.mapper.BookingDetailMapper;
import com.xinzhou.svcs.mobile.driver.mapper.BookingMapper;
import com.xinzhou.svcs.mobile.driver.service.IBookingDetailService;
import com.xinzhou.svcs.mobile.emp.entity.BoxOrder;
import com.xinzhou.svcs.mobile.emp.entity.ErrorOrder;
import com.xinzhou.svcs.mobile.emp.entity.OutOrder;
import com.xinzhou.svcs.mobile.emp.entity.QueueUp;
import com.xinzhou.svcs.mobile.emp.mapper.*;
import com.xinzhou.svcs.mobile.emp.service.*;
import com.xinzhou.svcs.mobile.emp.vo.OrderVO;
import com.xinzhou.svcs.mobile.register.entity.UserInfo;
import com.xinzhou.svcs.mobile.register.mapper.UserInfoMapper;
import com.xinzhou.svcs.pc.entity.*;
import com.xinzhou.svcs.pc.enums.FileRecordTypeEnum;
import com.xinzhou.svcs.pc.enums.TemplateTypeEnum;
import com.xinzhou.svcs.pc.mapper.*;
import com.xinzhou.svcs.pc.service.*;
```

```
import me.chanjar.weixin.mp.bean.template.WxMpTemplateData;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.*;

/**
 * <p>
 *
 * </p>
 *
 * @since 2021-03-01
 */
@Service
public class EmpScanServiceImpl implements IEmpScanService {
    @Autowired
    BookingMapper bookingMapper;
    @Autowired
    BookingDetailMapper bookingDetailMapper;
    @Autowired
    IFileUploadService fileUploadService;
    @Autowired
    UserInfoMapper userInfoMapper;
    @Autowired
    PushBaseMapper pushBaseMapper;
    @Autowired
    WxUtil wxUtil;
    @Autowired
    UploadUtil uploadUtil;
    @Autowired
    IQuayService quayService;
    @Autowired
    IDataDictionaryDetailService dictionaryDetailService;
    @Autowired
    QueueUpMapper queueUpMapper;
    @Autowired
    OnSiteVehicleMapper onSiteVehicleMapper;
    @Autowired
    InOrderMapper inOrderMapper;
    @Autowired
    BoxOrderMapper boxOrderMapper;
    @Autowired
    ErrorOrderMapper errorOrderMapper;
    @Autowired
    OutOrderMapper outOrderMapper;
    @Autowired
    InOrderDetailMapper inOrderDetailMapper;
    @Autowired
```

```
OutOrderDetailMapper outOrderDetailMapper;
@Autowired
ErrorOrderDetailMapper errorOrderDetailMapper;
@Autowired
BoxOrderDetailMapper boxOrderDetailMapper;
@Autowired
FileUploadMapper fileUploadMapper;
@Autowired
IUserSupplierService userSupplierService;
@Autowired
IOrderService orderService;
@Autowired
IDataDictionaryDetailService dictDetailService;
@Autowired
IBookingDetailService bookingDetailService;
@Autowired
IPushBaseService pushBaseService;
@Autowired
IInOrderService inOrderService;
@Autowired
IOutOrderService outOrderService;
@Autowired
IErrorOrderService errorOrderService;
@Autowired
IBoxOrderService boxOrderService;
/**
 *
 * @param bookingId
 */
@Override
@Transactional
public void queueUp(String bookingId) {
    Booking booking = bookingMapper.findBookingById(bookingId);
    if (booking.getBookingGrade() == 1) {
        throw new ServiceException("");
    }
    1.
    Date bookingTime = booking.getBookingTime();
    if (!DateUtil.isNow(bookingTime)) {
        throw new ServiceException(": " + DateUtil.getStrDate(bookingTime, null));
    }
    QueryWrapper<QueueUp> queryWrapper = new QueryWrapper<QueueUp>()
        .eq("IsDeleted", 0)
        .eq("BookingId", bookingId);
    List<QueueUp> list = queueUpMapper.selectList(queryWrapper);
    if (list != null && list.size() > 0) {
        throw new ServiceException(", ");
    }
    SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
    List<BookingDetail> bookingDetailList = booking.getBookingDetailList();
```

```

for (BookingDetail bookingDetail : bookingDetailList) {
    2. ()
    Date nowTime = new Date();
    Date startTime = null;
    Date endTime = null;
    String bookingBeginTime = bookingDetail.getBookingBeginTime();
    String bookingEndTime = bookingDetail.getBookingEndTime();
    try {

        Date parse = dateFormat.parse(DateUtil.getDate("yyyy-MM-dd") + " " + bookingEndTime + ":00");
        parse.setTime(parse.getTime() + 30 * 60 * 1000);
        String afterTime = dateFormat.format(parse);
        startTime = dateFormat.parse(DateUtil.getDate("yyyy-MM-dd") + " " + bookingBeginTime + ":00");
        endTime = dateFormat.parse(afterTime);
    } catch (ParseException e) {
        e.printStackTrace();
    }
    String quayNo = bookingDetail.getQuayNo();
    QueueUp queueUp = new QueueUp();
    queueUp.setBookingId(bookingDetail.getBookingId());
    queueUp.setBookingNo(bookingDetail.getBookingNo());
    queueUp.setBookingDetailId(bookingDetail.getId());
    queueUp.setVehicleNo(booking.getVehicleNo());
    queueUp.setBookingBeginTime(bookingDetail.getBookingBeginTime());
    queueUp.setBookingEndTime(bookingDetail.getBookingEndTime());
    queueUp.setQuayId(bookingDetail.getQuayId());
    queueUp.setQuayNo(bookingDetail.getQuayNo());
    queueUp.setQuayName(bookingDetail.getQuayName());
    queueUp.setCalledTimes(0);
    1: 2: 3:
    queueUp.setQueueStatus(1);
    queueUp.setOrderTime(new Date());
    ( < + )
    if (DateUtil.isEffectiveDate(nowTime, startTime, endTime)) {

        queryWrapper = new QueryWrapper<QueueUp>()
            .eq("quayNo", quayNo)
            .eq("BookingBeginTime", bookingBeginTime)
            .eq("BookingEndTime", bookingEndTime)
            .eq("CONVERT(varchar(100), CreationTime, 23)",
DateUtil.getDate("yyyy-MM-dd"));
        List queueUpList = queueUpMapper.selectList(queryWrapper);
        if (queueUpList == null || queueUpList.size() == 0) {

            queueUp.setSort(1);
        } else {

            queueUp.setSort(queueUpList.size() + 1);
        }
    } else {

```

```

        ( > +)
        queryWrapper = new QueryWrapper<QueueUp>()
            .eq("quayNo", quayNo)
            .eq("CONVERT(varchar(100), CreationTime, 23)", DateUtil.getDate("yyyy-MM-dd"))
            .orderByAsc("BookingBeginTime")
            .orderByAsc("Sort");
        List queueUpList = queueUpMapper.selectList(queryWrapper);
        if (queueUpList == null || queueUpList.size() == 0) {

            queueUp.setSort(1);
        } else {

            QueueUp lastQueueUp = (QueueUp)queueUpList.get(queueUpList.size() - 1);
            queueUp.setBookingBeginTime(lastQueueUp.getBookingBeginTime());
            queueUp.setBookingEndTime(lastQueueUp.getBookingEndTime());
            queueUp.setSort(lastQueueUp.getSort() + 1);
        }
    }
    int insert = queueUpMapper.insert(queueUp);
    if (insert != 1) {
        throw new InsertException(" ", "");
    }

    bookingDetail.setBookingStatus("2");
    bookingDetailMapper.updateById(bookingDetail);
}
}
/**
 *
 * @param orderVO
 * @return
 * 1. 2. 3. 4. 5.
 * 1. 2. 2.1 3. 4. 5.
 */
@Override
@Transactional
public void entranceAudit(OrderVO orderVO) {
    Booking booking = bookingMapper.findBookingById(orderVO.getBookingId());
    QueryWrapper<QueueUp> queryWrapper = null;
    if (booking.getBookingGrade() == 0) {
        queryWrapper = new QueryWrapper<QueueUp>()
            .eq("IsDeleted", 0)
            .eq("QueueStatus", 2)
            .eq("BookingId", orderVO.getBookingId());
        List<QueueUp> queueUpList = queueUpMapper.selectList(queryWrapper);
        if (queueUpList == null || queueUpList.size() == 0) {
            throw new ServiceException("");
        }
    }
}
1.

```

```

UpdateWrapper updateWrapper = new UpdateWrapper<Booking>()
    .eq("Id", orderV0.getBookingId())
    .set("IsIn", 1)
    .set("InTime", new Date())
    .set("InUserId", TokenUtil.getUserId());
bookingMapper.update(new Booking(), updateWrapper);

1. 2. 3. 4. 5.
1. 2. 2.1 3. 4. 5.
updateWrapper = new UpdateWrapper<BookingDetail>()
    .eq("IsDeleted", 0)
    .eq("BookingId", orderV0.getBookingId())
    .set("InTime", new Date())
    .set("BookingStatus", 4);
bookingDetailMapper.update(new BookingDetail(), updateWrapper);
if (booking.getBookingGrade() == 0) {
    2. 2 ( )
    queryWrapper = new QueryWrapper<QueueUp>()
        .eq("IsDeleted", 0)
        .eq("QueueStatus", 2)
        .eq("BookingId", orderV0.getBookingId())
        .orderByAsc("BookingBeginTime");
    QueueUp queueUp = queueUpMapper.selectList(queryWrapper).get(0);
    2.1 1: 2: 3: 4:
    queueUp.setQueueStatus(3);
    queueUpMapper.updateById(queueUp);
    2.2 (1., 2., 3.)
    quayService.switchQuayStatus(queueUp.getQuayId(), 2);
} else {
    quayService.switchQuayStatus(booking.getBookingDetailList().get(0).getQuayId(), 2);

    BookingDetail bookingDetail = booking.getBookingDetailList().get(0);
    if (bookingDetail.getForkCount() > 0) {

        UserInfo forkUser = userInfoMapper.selectById(bookingDetail.getForkUserId());
        ID
        UserInfo
        storeKeeper
        =
userSupplierService.findStoreKeeperBySupplierId(bookingDetail.getSupplierId());
        if ("1".equals(dictDetailService.getValueByDictCode("Sys_Mwsm"))) {
            pushBaseService.pushSysMsg(null, forkUser.getId(), forkUser.getName(),
                bookingDetail.getVehicleNo() + " , ", 1);
            pushBaseService.pushSysMsg(null, storeKeeper.getId(), storeKeeper.getName(),
                bookingDetail.getVehicleNo() + " , ", 1);
        }

        if ("1".equals(dictDetailService.getValueByDictCode("Urgent"))) {
            Map<String, String> dataMap = new HashMap<>();
            dataMap.put("thing6", bookingDetail.getQuayName());
            dataMap.put("car_number1", bookingDetail.getVehicleNo());

```

```

        wxUtil.sendWxSubscribeMsg(forkUser.getWeChatOpenId(),
TemplateTypeEnum.Urgent.getValue(), dataMap);
        wxUtil.sendWxSubscribeMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Urgent.getValue(), dataMap);
    }
}
}
3.
/( )

String path = dictionaryDetailService.getValueByDictName("UploadPath") +
String path = DateUtil.getDate("yyyyMMdd") + "/" + orderVO.getBookingNo() + "/";
fileUpload
if (orderVO.getFileList() != null && orderVO.getFileList().size() > 0) {
    uploadUtil.uploadFile(orderVO, path);
}
}
/**
 *
 * @param bookingId
 * @return
 */
@Override
public Booking findBookingAndOrder(String bookingId) {
    Booking booking = bookingMapper.findBookingById(bookingId);
    if (booking.getIsOut() == 1) {
        throw new ServiceException("");
    }
    QueryWrapper queryWrapper = new QueryWrapper<FileUpload>()
        .eq("IsDeleted", 0)
        .eq("BookingId", bookingId)
        .eq("FileRecordType", FileRecordTypeEnum.IN.getKey())
        .orderByAsc("CreationTime");
    List<FileUpload> fileUploadList = fileUploadMapper.selectList(queryWrapper);
    booking.setFileUploadList(fileUploadList);
    List<BookingDetail> bookingDetailList = booking.getBookingDetailList();
    for (BookingDetail bookingDetail : bookingDetailList) {
        String bookingDetailId = bookingDetail.getId();
        InOrder inOrder = inOrderMapper.findInOrder(bookingDetail.getId());
        if (inOrder != null) {
            List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "IN");
            inOrder.setPaperList(paperList);
        }
        bookingDetail.setInOrder(inOrder);
        BoxOrder boxOrder = boxOrderMapper.findBoxOrder(bookingDetail.getId());
        if (boxOrder != null) {
            List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "BOX");
            boxOrder.setPaperList(paperList);
        }
        bookingDetail.setBoxOrder(boxOrder);
    }
}

```

```

        OutOrder outOrder = outOrderMapper.findOutOrder(bookingDetail.getId());
        if (outOrder != null) {
            List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "OUT");
            outOrder.setPaperList(paperList);
        }
        bookingDetail.setOutOrder(outOrder);
        ErrorOrder errorOrder = errorOrderMapper.findErrorOrder(bookingDetail.getId());
        if (errorOrder != null) {
            List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "ERR");
            errorOrder.setPaperList(paperList);
            queryWrapper = new QueryWrapper<FileUpload>()
                .eq("IsDeleted", 0)
                .eq("FileRecordType", 3)
                .eq("BookingDetailId", bookingDetail.getId());
            List<FileUpload> fileList = fileUploadMapper.selectList(queryWrapper);
            errorOrder.setFileList(fileList);
        }
        bookingDetail.setErrorOrder(errorOrder);
    }
    return booking;
}
/**
 *
 * @param orderVO
 * @return
 */
@Override
public void leaveUpload(OrderVO orderVO) {
    /( )

    String path = dictionaryDetailService.getValueByDictName("UploadPath") +
    String path = DateUtil.getDate("yyyyMMdd") + "/" + orderVO.getBookingNo() + "/";
    fileUpload
    if (orderVO.getFileList() != null && orderVO.getFileList().size() > 0) {
        uploadUtil.uploadFile(orderVO, path);
    }
}
/**
 *
 * @param orderVO
 */
@Override
@Transactional
public void confirmLeave(OrderVO orderVO) {
    Booking booking = bookingMapper.findBookingById(orderVO.getBookingId());
    UserInfo driver = userInfoMapper.selectById(booking.getBookingUserId());
    for (BookingDetail bookingDetail : booking.getBookingDetailList()) {
        String bookingDetailId = bookingDetail.getId();
        InOrder inOrder = inOrderService.findInOrder(bookingDetailId);
        if (inOrder != null && inOrder.getOrderStatus() == null) {

```

```

        throw new ServiceException(", !");
    }
    if (inOrder != null && inOrder.getExamineFlag() == 1 && inOrder.getPaperList().size() == 0) {
        throw new ServiceException(", !");
    }
    ErrorOrder errorOrder = errorOrderService.findErrorOrder(bookingDetailId);
    if (errorOrder != null && errorOrder.getCheckStatus() != 2) {
        throw new ServiceException(", !");
    }
    if (errorOrder != null && errorOrder.getExamineFlag() == 1 && errorOrder.getPaperList().size() ==
0) {
        throw new ServiceException(", !");
    }
    BoxOrder boxOrder = boxOrderService.findBoxOrder(bookingDetailId);
    if (boxOrder != null && boxOrder.getCheckStatus() != 2) {
        throw new ServiceException(", !");
    }
    if (boxOrder != null && boxOrder.getExamineFlag() == 1 && boxOrder.getPaperList().size() == 0) {
        throw new ServiceException(", !");
    }
    OutOrder outOrder = outOrderService.findOutOrder(bookingDetailId);
    if (outOrder != null && outOrder.getCheckStatus() != 2) {
        throw new ServiceException(", !");
    }
    if (outOrder != null && outOrder.getExamineFlag() == 1 && outOrder.getPaperList().size() == 0) {
        throw new ServiceException(", !");
    }
    if (inOrder == null && errorOrder == null && outOrder == null && boxOrder == null) {
        throw new ServiceException("");
    }
}
1.
UpdateWrapper updateWrapper = new UpdateWrapper<Booking>()
    .eq("BookingNo", orderV0.getBookingNo())
    .set("IsOut", 1)
    .set("OutTime", new Date())
    .set("OutUserId", orderV0.getUserId())
    .set("Status", 4);
int update = bookingMapper.update(new Booking(), updateWrapper);
if (update != 1) throw new UpdateException(", ");
2.
updateWrapper = new UpdateWrapper<BookingDetail>()
    .eq("IsDeleted", 0)
    .eq("BookingNo", orderV0.getBookingNo())
    .set("BookingStatus", 5);
bookingDetailMapper.update(new BookingDetail(), updateWrapper);
3.
updateWrapper = new UpdateWrapper<OnSiteVehicle>()
    .eq("IsDeleted", 0)
    .eq("BookingId", orderV0.getBookingId())

```

```

        .set("Status", 4);
onSiteVehicleMapper.update(new OnSiteVehicle(), updateWrapper);
3.
/( )

String path = dictionaryDetailService.getValueByDictName("UploadPath") +
String path = DateUtil.getDate("yyyyMMdd") + "/" + orderVO.getBookingNo() + "/";
Map resultMap = new HashMap<String, String>();
    fileUpload
if (orderVO.getFileList() != null && orderVO.getFileList().size() > 0) {
    uploadUtil.uploadFile(orderVO, path);
}
}
/**
 *
 * @param pushBase
 *
 * ReceiveUserId ID ID
 * PushMessage
 * VehicleNo
 */
@Override
@Transactional
public void contactStoreKeeper(PushBase pushBase) {
    UserInfo userInfo = userInfoMapper.selectById(pushBase.getReceiveUserId());
    2.1
    pushBase.setSendUserId(TokenUtil.getUserId());
    pushBase.setSendUserName(TokenUtil.getUserName());
    pushBase.setReceiveUserId(userInfo.getId());
    pushBase.setReceiveUserName(userInfo.getName());
    pushBase.setPushDate(new Date());
    pushBase.setPushMessageType(1);
    int insert = pushBaseMapper.insert(pushBase);
    if (insert != 1) {
        throw new InsertException("");
    }
    2.2
    /*Map<String, String> dataMap = new HashMap<>();
    dataMap.put("car_number1", pushBase.getVehicleNo());
    dataMap.put("thing2", pushBase.getPushMessage());
    wxUtil.sendWxSubscribeMsg(userInfo.getWeChatOpenId(),
TemplateTypeEnum.ContactStoreKeeper.getValue(), dataMap);*/
    List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
    {{first.DATA}}
    : {{keyword1.DATA}}
    : {{keyword2.DATA}}
    {{remark.DATA}}
    wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
    wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", pushBase.getVehicleNo()));
    wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", pushBase.getPushMessage()));

```

```

        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(userInfo.getWeChatOpenId(),
TemplateTypeEnum.ContactStoreKeeper_Template.getValue(), wxMpTemplateDataList);
    }
    /**
     *
     * @param bookingId
     * @param isReplaceId
     */
    @Override
    public List<BookingDetail> storeKeeperScan(String bookingId, String isReplaceId) {
        String userId = StringUtils.isNotBlank(isReplaceId) ? isReplaceId : TokenUtil.getUserId();
        List<BookingDetail> bookingDetailList = bookingMapper.findDetailByStoreKeeper(bookingId, userId);
        if (bookingDetailList == null || bookingDetailList.size() == 0) {
            throw new ServiceException("");
        }
        List<String> detailIdList = new ArrayList<>();
        boolean updateFlag = false;
        for (BookingDetail bookingDetail : bookingDetailList) {
            /*if (!"4".equals(bookingDetail.getBookingStatus())) {
                throw new ServiceException("");
            }*/
            detailIdList.add(bookingDetail.getId());
            if (bookingDetail.getScanTime() == null) {
                bookingDetail.setScanTime(new Date());
                updateFlag = true;
            }
            QueryWrapper queryWrapper = null;
            InOrder inOrder = inOrderMapper.findInOrder(bookingDetail.getId());
            if (inOrder != null) {
                List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "IN");
                inOrder.setPaperList(paperList);
            }
            bookingDetail.setInOrder(inOrder);
            queryWrapper = new QueryWrapper<InOrderDetail>()
                .eq("IsDeleted", 0)
                .eq("BookingDetailId", bookingDetail.getId())
                .orderByAsc("Sort");
            List<InOrderDetail> inOrderDetailList = inOrderDetailMapper.selectList(queryWrapper);
            bookingDetail.setInOrderDetailList(inOrderDetailList);
            BoxOrder boxOrder = boxOrderMapper.findBoxOrder(bookingDetail.getId());
            if (boxOrder != null) {
                List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "BOX");
                boxOrder.setPaperList(paperList);
            }
            bookingDetail.setBoxOrder(boxOrder);
            OutOrder outOrder = outOrderMapper.findOutOrder(bookingDetail.getId());
            if (outOrder != null) {
                List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "OUT");
                outOrder.setPaperList(paperList);
            }
        }
    }

```

```
    }
    bookingDetail.setOutOrder(outOrder);
    ErrorOrder errorOrder = errorOrderMapper.findErrorOrder(bookingDetail.getId());
    if (errorOrder != null) {
        List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "ERR");
        errorOrder.setPaperList(paperList);
        queryWrapper = new QueryWrapper<FileUpload>()
            .eq("IsDeleted", 0)
            .eq("FileRecordType", 3)
            .eq("BookingDetailId", bookingDetail.getId());
        List<FileUpload> fileUploadList = fileUploadMapper.selectList(queryWrapper);
        errorOrder.setFileList(fileUploadList);
    }
    bookingDetail.setErrorOrder(errorOrder);
}
if (updateFlag) {
    bookingDetailService.updateBatchById(bookingDetailList);
    /* 1: 2: 3: 4:
    UpdateWrapper<QueueUp> updateWrapper = new UpdateWrapper<QueueUp>()
        .eq("IsDeleted", 0)
        .in("BookingDetailId", detailIdList)
        .set("QueueStatus", 3);
    queueUpMapper.update(new QueueUp(), updateWrapper);*/
}
return bookingDetailList;
}
}

package com.xinzhou.svcs.mobile.emp.service.impl;
import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.conditions.update.UpdateWrapper;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xinzhou.svcs.common.config.DataInitialize;
import com.xinzhou.svcs.common.ex.InsertException;
import com.xinzhou.svcs.common.ex.UpdateException;
import com.xinzhou.svcs.common.utils.OrderUtil;
import com.xinzhou.svcs.common.utils.TokenUtil;
import com.xinzhou.svcs.common.utils.WxUtil;
import com.xinzhou.svcs.mobile.driver.mapper.BookingDetailMapper;
import com.xinzhou.svcs.mobile.emp.entity.*;
import com.xinzhou.svcs.mobile.emp.mapper.*;
import com.xinzhou.svcs.mobile.emp.service.IErrorOrderDetailService;
import com.xinzhou.svcs.mobile.emp.service.IErrorOrderService;
import com.xinzhou.svcs.mobile.emp.service.IExamineService;
import com.xinzhou.svcs.mobile.emp.service.IOrderService;
import com.xinzhou.svcs.mobile.register.entity.UserInfo;
import com.xinzhou.svcs.mobile.register.mapper.UserInfoMapper;
import com.xinzhou.svcs.mobile.register.service.IUserInfoService;
import com.xinzhou.svcs.pc.entity.FileUpload;
import com.xinzhou.svcs.pc.enums.TemplateTypeEnum;
import com.xinzhou.svcs.pc.mapper.FileUploadMapper;
```

```
import com.xinzhou.svcs.pc.service.IDataDictionaryDetailService;
import com.xinzhou.svcs.pc.service.IFileUploadService;
import com.xinzhou.svcs.pc.service.IPushBaseService;
import me.chanjar.weixin.mp.bean.template.WxMpTemplateData;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
/**
 * <p>
 *
 * </p>
 *
 * @since 2021-03-09
 */
@Service
public class ErrorOrderServiceImpl extends ServiceImpl<ErrorOrderMapper, ErrorOrder> implements
IErrorOrderService {
    @Autowired
    ErrorOrderMapper mapper;
    @Autowired
    ErrorOrderDetailMapper detailMapper;
    @Autowired
    IErrorOrderDetailService detailService;
    @Autowired
    ExamineDetailMapper examineDetailMapper;
    @Autowired
    ExamineRecordMapper examineRecordMapper;
    @Autowired
    OrderMapper orderMapper;
    @Autowired
    FileUploadMapper fileUploadMapper;
    @Autowired
    UserInfoMapper userInfoMapper;
    @Autowired
    IUserInfoService userInfoService;
    @Autowired
    WxUtil wxUtil;
    @Autowired
    IOrderService orderService;
    @Autowired
    BookingDetailMapper bookingDetailMapper;
    @Autowired
    IDataDictionaryDetailService dictDetailService;
    @Autowired
    IPushBaseService pushBaseService;
```

```

@Autowired
IExamineService examineService;
@Autowired
IFileUploadService fileUploadService;
/**
 *
 * @param errorOrderList
 */
@Override
@Transactional
public void addErrorOrder(List<ErrorOrder> errorOrderList) {
    String examineId = DataInitialize.examineMap.get("");
    ExamineDetail examineDetail = examineDetailMapper.findByExamineId(examineId).get(0);
    for (ErrorOrder errorOrder : errorOrderList) {
        UpdateWrapper updateWrapper = new UpdateWrapper<ErrorOrderDetail>()
            .eq("IsDeleted", 0)
            .eq("BookingDetailId", errorOrder.getBookingDetailId())
            .set("IsDeleted", 1);
        detailMapper.update(new ErrorOrderDetail(), updateWrapper);
        updateWrapper = new UpdateWrapper<ErrorOrder>()
            .eq("IsDeleted", 0)
            .eq("BookingDetailId", errorOrder.getBookingDetailId())
            .set("IsDeleted", 1);
        mapper.update(new ErrorOrder(), updateWrapper);
        String orderNo = OrderUtil.genOrderNo("ERR", errorOrder.getSupplierNo());
        errorOrder.setOrderNo(orderNo);
        if (errorOrder.getExamineFlag() != null && errorOrder.getExamineFlag() == 1) {
            errorOrder.setCheckStatus(2);
        } else {
            errorOrder.setCheckStatus(1);
        }
    }
    1.
    int insert = mapper.insert(errorOrder);
    if (insert != 1) {
        throw new InsertException("");
    }
    2.
    List<ErrorOrderDetail> errorOrderDetailList = errorOrder.getErrorOrderDetailList();
    for (ErrorOrderDetail errorOrderDetail : errorOrderDetailList) {
        errorOrderDetail.setOrderId(errorOrder.getId());
        errorOrderDetail.setOrderNo(orderNo);
        errorOrderDetail.setBookingDetailId(errorOrder.getBookingDetailId());
        errorOrderDetail.setBookingNo(errorOrder.getBookingNo());
        errorOrderDetail.setSupplierId(errorOrder.getSupplierId());
        errorOrderDetail.setSupplierNo(errorOrder.getSupplierNo());
        errorOrderDetail.setSupplierName(errorOrder.getSupplierName());
    }
    boolean b = detailService.saveBatch(errorOrderDetailList);
    if (!b) {
        throw new InsertException("");
    }
}

```

```

    }
    3.
    if (errorOrder.getExamineFlag() != null && errorOrder.getExamineFlag() == 0) {
        ExamineRecord examineRecord = new ExamineRecord();
        examineRecord.setOrderId(errorOrder.getId());
        examineRecord.setOrderNo(orderNo);
        1., 2., 3., 4.
        examineRecord.setExamineType(3);
        examineRecord.setRoleCode(examineDetail.getRoleCode());
        examineRecord.setRoleName(examineDetail.getRoleName());
        examineRecord.setSort(1);
        examineRecord.setExamineId(examineDetail.getExamineId());ID
        examineRecord.setExamineDetailId(examineDetail.getId());ID
        examineRecord.setCheckStatus(1);1. 2. 3.

        UserInfo examineUser = examineService.getNextExamineUser(examineRecord, examineDetail);
        examineRecord.setExamineUserId(examineUser.getId());
        examineRecord.setExamineUserName(examineUser.getName());
        int rows = examineRecordMapper.insert(examineRecord);
        if(rows != 1){
            throw new InsertException("");
        }

        if ("1".equals(dictDetailService.getValueByDictCode("Sys_Yclj_Xj"))) {
            pushBaseService.pushSysMsg(null, examineUser.getId(), examineUser.getName(),
                "", 1);
        }

        if ("1".equals(dictDetailService.getValueByDictCode("OrderNext"))) {
            /*Map<String, String> dataMap = new HashMap<>();
            dataMap.put("thing2", "");
            dataMap.put("thing3", "");
            wxUtil.sendWxSubscribeMsg(examineUser.getWeChatOpenId(),
            TemplateTypeEnum.OrderNext.getValue(), dataMap);*/
            List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
            {{first.DATA}}
            : {{keyword1.DATA}}
            : {{keyword2.DATA}}
            : {{keyword3.DATA}} initiator
            : {{keyword4.DATA}} submitter
            : {{keyword5.DATA}}
            {{remark.DATA}}
            UserInfo storeKeeper = userInfoMapper.selectById(TokenUtil.getUserId());
            wxMpTemplateDataList.add(new WxMpTemplateData("first", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", errorOrder.getOrderNo()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword3", storeKeeper.getName()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword4", storeKeeper.getName()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword5", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));

```

```

        wxUtil.sendWxTemplateMsg(examineUser.getWeChatOpenId(),
TemplateTypeEnum.OrderNext_Template.getValue(), wxMpTemplateDataList);
    }
}
}
}
/**
 *
 * @param bookingDetailId
 */
@Override
@Transactional
public void deleteErrorOrder(String bookingDetailId) {
    UpdateWrapper updateWrapper = new UpdateWrapper<ErrorOrderDetail>()
        .eq("IsDeleted", 0)
        .eq("BookingDetailId", bookingDetailId)
        .set("IsDeleted", 1);
    detailMapper.update(new ErrorOrderDetail(), updateWrapper);
    updateWrapper = new UpdateWrapper<ErrorOrder>()
        .eq("IsDeleted", 0)
        .eq("BookingDetailId", bookingDetailId)
        .set("IsDeleted", 1);
    mapper.update(new ErrorOrder(), updateWrapper);
}
/**
 *
 * @param errorOrderId
 */
@Override
public void confirmErrorOrder(String errorOrderId) {
    UpdateWrapper<ErrorOrder> updateWrapper =
        new UpdateWrapper<ErrorOrder>().eq("Id", errorOrderId).set("OrderStatus", 1);
    int update = mapper.update(new ErrorOrder(), updateWrapper);
    if (update != 1) throw new UpdateException(", ");
}
/**
 *
 * @param isReplaceId
 * @return
 */
@Override
public List<ErrorOrder> findErrorOrderList(String isReplaceId) {
    String userId = StringUtils.isNotBlank(isReplaceId) ? isReplaceId : TokenUtil.getUserId();
    List<ErrorOrder> errorOrderList = mapper.findErrorOrderList(userId);
    return errorOrderList;
}
/**
 * ID
 * @param bookingDetailId
 * @return

```

```

    */
    @Override
    public ErrorOrder findErrorOrder(String bookingDetailId) {
        ErrorOrder errorOrder = mapper.findErrorOrder(bookingDetailId);
        if (errorOrder != null) {
            List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetailId, "ERR");
            errorOrder.setPaperList(paperList);
            QueryWrapper queryWrapper = new QueryWrapper<FileUpload>()
                .eq("IsDeleted", 0)
                .eq("FileRecordType", 3)
                .eq("BookingDetailId", bookingDetailId);
            List<FileUpload> fileUploadList = fileUploadMapper.selectList(queryWrapper);
            errorOrder.setFileList(fileUploadList);
            UserInfo storeKeeper = orderService.getStoreKeeper(errorOrder.getSupplierId());
            String driver = bookingDetailMapper.selectById(bookingDetailId).getBookingUserName();
            UserInfo monitor = userInfoService.findMonitor();
            errorOrder.setStoreKeeper(storeKeeper.getName());
            errorOrder.setDriver(driver);
            errorOrder.setMonitor(monitor.getName());
        }
        return errorOrder;
    }
}

package com.xinzhou.svcs.mobile.emp.service.impl;
import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.conditions.update.UpdateWrapper;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xinzhou.svcs.common.ex.ServiceException;
import com.xinzhou.svcs.common.utils.DateUtil;
import com.xinzhou.svcs.common.utils.TokenUtil;
import com.xinzhou.svcs.common.utils.WxUtil;
import com.xinzhou.svcs.mobile.driver.entity.Booking;
import com.xinzhou.svcs.mobile.driver.entity.BookingDetail;
import com.xinzhou.svcs.mobile.driver.mapper.BookingDetailMapper;
import com.xinzhou.svcs.mobile.driver.mapper.BookingMapper;
import com.xinzhou.svcs.mobile.emp.entity.*;
import com.xinzhou.svcs.mobile.emp.mapper.*;
import com.xinzhou.svcs.mobile.emp.service.IExamineDetailService;
import com.xinzhou.svcs.mobile.emp.service.IExamineService;
import com.xinzhou.svcs.mobile.register.entity.UserInfo;
import com.xinzhou.svcs.mobile.register.mapper.UserInfoMapper;
import com.xinzhou.svcs.mobile.register.service.IUserInfoService;
import com.xinzhou.svcs.pc.enums.RolesEnum;
import com.xinzhou.svcs.pc.enums.TemplateTypeEnum;
import com.xinzhou.svcs.pc.mapper.PushBaseMapper;
import com.xinzhou.svcs.pc.service.IDataDictionaryDetailService;
import com.xinzhou.svcs.pc.service.IOperateRecordService;
import com.xinzhou.svcs.pc.service.IPushBaseService;
import com.xinzhou.svcs.pc.service.IUserSupplierService;
import me.chanjar.weixin.mp.api.WxMpService;

```

```
import me.chanjar.weixin.mp.bean.template.WxMpTemplateData;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.*;

/**
 * <p>
 *
 * </p>
 *
 * @since 2021-03-01
 */
@Service
public class ExamineServiceImpl extends ServiceImpl<ExamineMapper, Examine> implements IExamineService {
    @Autowired
    ExamineMapper examineMapper;
    @Autowired
    ExamineRecordMapper recordMapper;
    @Autowired
    ExamineDetailMapper examineDetailMapper;
    @Autowired
    WxMpService wxMpService;
    @Autowired
    UserInfoMapper userInfoMapper;
    @Autowired
    IUserInfoService userInfoService;
    @Autowired
    BookingMapper bookingMapper;
    @Autowired
    BookingDetailMapper bookingDetailMapper;
    @Autowired
    ErrorOrderMapper errorOrderMapper;
    @Autowired
    OutOrderMapper outOrderMapper;
    @Autowired
    BoxOrderMapper boxOrderMapper;
    @Autowired
    WxUtil wxUtil;
    @Autowired
    IExamineDetailService examineDetailService;
    @Autowired
    PushBaseMapper pushBaseMapper;
    @Autowired
    IPushBaseService pushBaseService;
    @Autowired
    IOperateRecordService operateRecordService;
    @Autowired
    IUserSupplierService userSupplierService;
    @Autowired
    IDataDictionaryDetailService detailService;
```

```

/**
 *
 * @return
 */
@Override
public List<Examine> findList() {
    QueryWrapper<Examine> queryWrapper = new QueryWrapper<>();
    queryWrapper.eq("IsDeleted", 0);
    List<Examine> examineList = examineMapper.selectList(queryWrapper);
    return examineList;
}

wxUtil.sendWxTemplateMsg(examineUser.getWeChatOpenId(),
TemplateTypeEnum.OrderNext_Template.getValue(), wxMpTemplateDataList);
}
}
}
}
/**
 *
 * @param bookingDetailId
 */
@Override
@Transactional
public void deleteErrorOrder(String bookingDetailId) {
    UpdateWrapper updateWrapper = new UpdateWrapper<ErrorOrderDetail>()
        .eq("IsDeleted", 0)
        .eq("BookingDetailId", bookingDetailId)
        .set("IsDeleted", 1);
    detailMapper.update(new ErrorOrderDetail(), updateWrapper);
    updateWrapper = new UpdateWrapper<ErrorOrder>()
        .eq("IsDeleted", 0)
        .eq("BookingDetailId", bookingDetailId)
        .set("IsDeleted", 1);
    mapper.update(new ErrorOrder(), updateWrapper);
}
/**
 * ID
 * @param examineId
 * @return
 */
@Override
public List<ExamineDetail> getDetailByExamineId(String examineId) {
    QueryWrapper<ExamineDetail> queryWrapper = new QueryWrapper<ExamineDetail>()
        .eq("IsDeleted", 0)
        .eq("ExamineId", examineId)
        .orderByAsc("Sort");
    List<ExamineDetail> examineList = examineDetailMapper.selectList(queryWrapper);
    return examineList;
}
}

```

```

    *
    * @param orderId
    * @return
    */
@Override
public List<ExamineDetail> getExamineDetailList(String orderId) {
    QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("OrderId", orderId)
        .eq("Sort", 1);
    ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
    List examineDetailList = examineMapper.getExamineDetailList(examineRecord.getExamineId());
    return examineDetailList;
}
/**
 *
 * @param examineDetailList
 */
@Override
@Transactional
public void saveDetail(List<ExamineDetail> examineDetailList) {
    1.
    QueryWrapper queryWrapper = new QueryWrapper<ExamineDetail>()
        .eq("IsDeleted", 0)
        .eq("ExamineId", examineDetailList.get(0).getExamineId())
        .orderByAsc("Sort");
    List oldList = examineDetailMapper.selectList(queryWrapper);
    queryWrapper = new QueryWrapper<ExamineDetail>()
        .eq("IsDeleted", 0)
        .eq("ExamineId", examineDetailList.get(0).getExamineId());
    examineDetailMapper.delete(queryWrapper);
    2.
    examineDetailService.saveBatch(examineDetailList);
    operateRecordService.insertRecord("", "-", ": " + oldList.toString() + ",: " +
examineDetailList.toString());
}

}

2.2
/*Map<String, String> dataMap = new HashMap<>();
dataMap.put("car_number1", pushBase.getVehicleNo());
dataMap.put("thing2", pushBase.getPushMessage());
wxUtil.sendWxSubscribeMsg(userInfo.getWeChatOpenId(),
TemplateTypeEnum.ContactStoreKeeper.getValue(), dataMap);*/
List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
{{first.DATA}}
: {{keyword1.DATA}}
: {{keyword2.DATA}}
{{remark.DATA}}
wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", pushBase.getVehicleNo()));
wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", pushBase.getPushMessage()));

```

```

        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(userInfo.getWeChatOpenId(),
TemplateTypeEnum.ContactStoreKeeper_Template.getValue(), wxMpTemplateDataList);
    }
    /**
     *
     * @param bookingId
     * @param isReplaceId
     */
    @Override
    public List<BookingDetail> storeKeeperScan(String bookingId, String isReplaceId) {
        String userId = StringUtils.isNotBlank(isReplaceId) ? isReplaceId : TokenUtil.getUserId();
        List<BookingDetail> bookingDetailList = bookingMapper.findDetailByStoreKeeper(bookingId, userId);
        if (bookingDetailList == null || bookingDetailList.size() == 0) {
            throw new ServiceException("");
        }
        List<String> detailIdList = new ArrayList<>();
        boolean updateFlag = false;
        for (BookingDetail bookingDetail : bookingDetailList) {
            /*if (!"4".equals(bookingDetail.getBookingStatus())) {
                throw new ServiceException("");
            }*/
            detailIdList.add(bookingDetail.getId());
            if (bookingDetail.getScanTime() == null) {
                bookingDetail.setScanTime(new Date());
                updateFlag = true;
            }
            QueryWrapper queryWrapper = null;
            InOrder inOrder = inOrderMapper.findInOrder(bookingDetail.getId());
            if (inOrder != null) {
                List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "IN");
                inOrder.setPaperList(paperList);
            }
            bookingDetail.setInOrder(inOrder);
            queryWrapper = new QueryWrapper<InOrderDetail>()
                .eq("IsDeleted", 0)
                .eq("BookingDetailId", bookingDetail.getId())
                .orderByAsc("Sort");
            List<InOrderDetail> inOrderDetailList = inOrderDetailMapper.selectList(queryWrapper);
            bookingDetail.setInOrderDetailList(inOrderDetailList);
            BoxOrder boxOrder = boxOrderMapper.findBoxOrder(bookingDetail.getId());
            if (boxOrder != null) {
                List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(),
"BOX");

                boxOrder.setPaperList(paperList);
            }
            bookingDetail.setBoxOrder(boxOrder);
            OutOrder outOrder = outOrderMapper.findOutOrder(bookingDetail.getId());
            if (outOrder != null) {

```

```

        List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(),
"OUT");

        outOrder.setPaperList(paperList);
    }
    bookingDetail.setOutOrder(outOrder);
    ErrorOrder errorOrder = errorOrderMapper.findErrorOrder(bookingDetail.getId());
    if (errorOrder != null) {
        List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(),
"ERR");

        errorOrder.setPaperList(paperList);
        queryWrapper = new QueryWrapper<FileUpload>()
            .eq("IsDeleted", 0)
            .eq("FileRecordType", 3)
            .eq("BookingDetailId", bookingDetail.getId());
        List<FileUpload> fileUploadList = fileUploadMapper.selectList(queryWrapper);
        errorOrder.setFileList(fileUploadList);
    }
    bookingDetail.setErrorOrder(errorOrder);
}
if (updateFlag) {
    bookingDetailService.updateBatchById(bookingDetailList);
    /* 1: 2: 3: 4:
    UpdateWrapper<QueueUp> updateWrapper = new UpdateWrapper<QueueUp>()
        .eq("IsDeleted", 0)
        .in("BookingDetailId", detailIdList)
        .set("QueueStatus", 3);
    queueUpMapper.update(new QueueUp(), updateWrapper);*/
}
return bookingDetailList;
}
}

package com.xinzhou.svcs.mobile.emp.service.impl;
import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.conditions.update.UpdateWrapper;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xinzhou.svcs.common.config.DataInitialize;
import com.xinzhou.svcs.common.ex.InsertException;
import com.xinzhou.svcs.common.ex.UpdateException;
import com.xinzhou.svcs.common.utils.OrderUtil;
import com.xinzhou.svcs.common.utils.TokenUtil;
import com.xinzhou.svcs.common.utils.WxUtil;
import com.xinzhou.svcs.mobile.driver.mapper.BookingDetailMapper;
import com.xinzhou.svcs.mobile.emp.entity.*;
import com.xinzhou.svcs.mobile.emp.mapper.*;
import com.xinzhou.svcs.mobile.emp.service.IErrorOrderDetailService;
import com.xinzhou.svcs.mobile.emp.service.IErrorOrderService;
import com.xinzhou.svcs.mobile.emp.service.IExamineService;
import com.xinzhou.svcs.mobile.emp.service.IOrderService;
import com.xinzhou.svcs.mobile.register.entity.UserInfo;
import com.xinzhou.svcs.mobile.register.mapper.UserInfoMapper;

```

```

import com.xinzhou.svcs.mobile.register.service.IUserInfoService;
import com.xinzhou.svcs.pc.entity.FileUpload;
import com.xinzhou.svcs.pc.enums.TemplateTypeEnum;
import com.xinzhou.svcs.pc.mapper.FileUploadMapper;
import com.xinzhou.svcs.pc.service.IDataDictionaryDetailService;
import com.xinzhou.svcs.pc.service.IFileUploadService;
import com.xinzhou.svcs.pc.service.IPushBaseService;
import me.chanjar.weixin.mp.bean.template.WxMpTemplateData;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
/**
 * <p>
 *
 * </p>
 *
 * @since 2021-03-09
 */
@Service
public class ErrorOrderServiceImpl extends ServiceImpl<ErrorOrderMapper, ErrorOrder> implements
IErrorOrderService {
    @Autowired
    ErrorOrderMapper mapper;
    @Autowired
    ErrorOrderDetailMapper detailMapper;
    @Autowired
    IErrorOrderDetailService detailService;
    @Autowired
    ExamineDetailMapper examineDetailMapper;
    @Autowired
    ExamineRecordMapper examineRecordMapper;
    @Autowired
    OrderMapper orderMapper;
    @Autowired
    FileUploadMapper fileUploadMapper;
    @Autowired
    UserInfoMapper userInfoMapper;
    @Autowired
    IUserInfoService userInfoService;
    @Autowired
    WxUtil wxUtil;
    @Autowired
    IOrderService orderService;
    @Autowired
    BookingDetailMapper bookingDetailMapper;

```

```

@Autowired
IDataDictionaryDetailsService dictDetailsService;
@Autowired
IPushBaseService pushBaseService;
@Autowired
IExamineService examineService;
@Autowired
IFileUploadService fileUploadService;
/**
 *
 * @param errorOrderList
 */
@Override
@Transactional
public void addErrorOrder(List<ErrorOrder> errorOrderList) {
    String examineId = DataInitialize.examineMap.get("");
    ExamineDetail examineDetail = examineDetailMapper.findByExamineId(examineId).get(0);
    for (ErrorOrder errorOrder : errorOrderList) {
        UpdateWrapper updateWrapper = new UpdateWrapper<ErrorOrderDetail>()
            .eq("IsDeleted", 0)
            .eq("BookingDetailId", errorOrder.getBookingDetailId())
            .set("IsDeleted", 1);
        detailMapper.update(new ErrorOrderDetail(), updateWrapper);
        updateWrapper = new UpdateWrapper<ErrorOrder>()
            .eq("IsDeleted", 0)
            .eq("BookingDetailId", errorOrder.getBookingDetailId())
            .set("IsDeleted", 1);
        mapper.update(new ErrorOrder(), updateWrapper);
        String orderNo = OrderUtil.genOrderNo("ERR", errorOrder.getSupplierNo());
        errorOrder.setOrderNo(orderNo);
        if (errorOrder.getExamineFlag() != null && errorOrder.getExamineFlag() == 1) {
            errorOrder.setCheckStatus(2);
        } else {
            errorOrder.setCheckStatus(1);
        }
        1.
        int insert = mapper.insert(errorOrder);
        if (insert != 1) {
            throw new InsertException("");
        }
        2.
        List<ErrorOrderDetail> errorOrderDetailList = errorOrder.getErrorOrderDetailList();
        for (ErrorOrderDetail errorOrderDetail : errorOrderDetailList) {
            errorOrderDetail.setOrderId(errorOrder.getId());
            errorOrderDetail.setOrderNo(orderNo);
            errorOrderDetail.setBookingDetailId(errorOrder.getBookingDetailId());
            errorOrderDetail.setBookingNo(errorOrder.getBookingNo());
            errorOrderDetail.setSupplierId(errorOrder.getSupplierId());
            errorOrderDetail.setSupplierNo(errorOrder.getSupplierNo());
            errorOrderDetail.setSupplierName(errorOrder.getSupplierName());
        }
    }
}

```

```

    }
    boolean b = detailService.saveBatch(errorOrderDetailList);
    if (!b) {
        throw new InsertException("");
    }
    3.
    if (errorOrder.getExamineFlag() != null && errorOrder.getExamineFlag() == 0) {
        ExamineRecord examineRecord = new ExamineRecord();
        examineRecord.setOrderId(errorOrder.getId());
        examineRecord.setOrderNo(orderNo);
        1., 2., 3., 4.
        examineRecord.setExamineType(3);
        examineRecord.setRoleCode(examineDetail.getRoleCode());
        examineRecord.setRoleName(examineDetail.getRoleName());
        examineRecord.setSort(1);
        examineRecord.setExamineId(examineDetail.getExamineId());ID
        examineRecord.setExamineDetailId(examineDetail.getId());ID
        examineRecord.setCheckStatus(1);1. 2. 3.

        UserInfo examineUser = examineService.getNextExamineUser(examineRecord, examineDetail);
        examineRecord.setExamineUserId(examineUser.getId());
        examineRecord.setExamineUserName(examineUser.getName());
        int rows = examineRecordMapper.insert(examineRecord);
        if (rows != 1) {
            throw new InsertException("");
        }

        if ("1".equals(dictDetailService.getValueByDictCode("Sys_Yclj_Xj"))) {
            pushBaseService.pushSysMsg(null, examineUser.getId(), examineUser.getName(),
                "", 1);
        }

        if ("1".equals(dictDetailService.getValueByDictCode("OrderNext"))) {
            /*Map<String, String> dataMap = new HashMap<>();
            dataMap.put("thing2", "");
            dataMap.put("thing3", "");
            wxUtil.sendWxSubscribeMsg(examineUser.getWeChatOpenId(),
TemplateTypeEnum.OrderNext.getValue(), dataMap);*/
            List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
            {{first.DATA}}
            : {{keyword1.DATA}}
            : {{keyword2.DATA}}
            : {{keyword3.DATA}} initiator
            : {{keyword4.DATA}} submitter
            : {{keyword5.DATA}}
            {{remark.DATA}}
            UserInfo storeKeeper = userInfoMapper.selectById(TokenUtil.getUserId());
            wxMpTemplateDataList.add(new WxMpTemplateData("first", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", errorOrder.getOrderNo()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword3", storeKeeper.getName()));

```

```

        wxMpTemplateDataList.add(new WxMpTemplateData("keyword4", storeKeeper.getName()));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword5", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(examineUser.getWeChatOpenId(),
TemplateTypeEnum.OrderNext_Template.getValue(), wxMpTemplateDataList);
    }
}

/**
 *
 * @param bookingDetailId
 */
@Override
@Transactional
public void deleteErrorOrder(String bookingDetailId) {
    UpdateWrapper updateWrapper = new UpdateWrapper<ErrorOrderDetail>()
        .eq("IsDeleted", 0)
        .eq("BookingDetailId", bookingDetailId)
        .set("IsDeleted", 1);
    detailMapper.update(new ErrorOrderDetail(), updateWrapper);
    updateWrapper = new UpdateWrapper<ErrorOrder>()
        .eq("IsDeleted", 0)
        .eq("BookingDetailId", bookingDetailId)
        .set("IsDeleted", 1);
    mapper.update(new ErrorOrder(), updateWrapper);
}

/**
 *
 * @param errorOrderId
 */
@Override
public void confirmErrorOrder(String errorOrderId) {
    UpdateWrapper<ErrorOrder> updateWrapper =
        new UpdateWrapper<ErrorOrder>().eq("Id", errorOrderId).set("OrderStatus", 1);
    int update = mapper.update(new ErrorOrder(), updateWrapper);
    if (update != 1) throw new UpdateException(", ");
}

/**
 *
 * @param isReplaceId
 * @return
 */
@Override
public List<ErrorOrder> findErrorOrderList(String isReplaceId) {
    String userId = StringUtils.isNotBlank(isReplaceId) ? isReplaceId : TokenUtil.getUserId();
    List<ErrorOrder> errorOrderList = mapper.findErrorOrderList(userId);
    return errorOrderList;
}

/**

```

```

    * ID
    * @param bookingDetailId
    * @return
    */
    @Override
    public ErrorOrder findErrorOrder(String bookingDetailId) {
        ErrorOrder errorOrder = mapper.findErrorOrder(bookingDetailId);
        if (errorOrder != null) {
            List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetailId, "ERR");
            errorOrder.setPaperList(paperList);
            QueryWrapper queryWrapper = new QueryWrapper<FileUpload>()
                .eq("IsDeleted", 0)
                .eq("FileRecordType", 3)
                .eq("BookingDetailId", bookingDetailId);
            List<FileUpload> fileUploadList = fileUploadMapper.selectList(queryWrapper);
            errorOrder.setFileList(fileUploadList);
            UserInfo storeKeeper = orderService.getStoreKeeper(errorOrder.getSupplierId());
            String driver = bookingDetailMapper.selectById(bookingDetailId).getBookingUserName();
            UserInfo monitor = userInfoService.findMonitor();
            errorOrder.setStoreKeeper(storeKeeper.getName());
            errorOrder.setDriver(driver);
            errorOrder.setMonitor(monitor.getName());
        }
        return errorOrder;
    }
}

package com.xinzhou.svcs.mobile.emp.service.impl;
import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.conditions.update.UpdateWrapper;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xinzhou.svcs.common.ex.ServiceException;
import com.xinzhou.svcs.common.utils.DateUtil;
import com.xinzhou.svcs.common.utils.TokenUtil;
import com.xinzhou.svcs.common.utils.WxUtil;
import com.xinzhou.svcs.mobile.driver.entity.Booking;
import com.xinzhou.svcs.mobile.driver.entity.BookingDetail;
import com.xinzhou.svcs.mobile.driver.mapper.BookingDetailMapper;
import com.xinzhou.svcs.mobile.driver.mapper.BookingMapper;
import com.xinzhou.svcs.mobile.emp.entity.*;
import com.xinzhou.svcs.mobile.emp.mapper.*;
import com.xinzhou.svcs.mobile.emp.service.IExamineDetailService;
import com.xinzhou.svcs.mobile.emp.service.IExamineService;
import com.xinzhou.svcs.mobile.register.entity.UserInfo;
import com.xinzhou.svcs.mobile.register.mapper.UserInfoMapper;
import com.xinzhou.svcs.mobile.register.service.IUserInfoService;
import com.xinzhou.svcs.pc.enums.RolesEnum;
import com.xinzhou.svcs.pc.enums.TemplateTypeEnum;
import com.xinzhou.svcs.pc.mapper.PushBaseMapper;
import com.xinzhou.svcs.pc.service.IDataDictionaryDetailService;
import com.xinzhou.svcs.pc.service.IOperateRecordService;

```

```

import com.xinzhou.svcs.pc.service.IPushBaseService;
import com.xinzhou.svcs.pc.service.IUserSupplierService;
import me.chanjar.weixin.mp.api.WxMpService;
import me.chanjar.weixin.mp.bean.template.WxMpTemplateData;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.*;

/**
 * <p>
 *
 * </p>
 *
 * @since 2021-03-01
 */
@Service
public class ExamineServiceImpl extends ServiceImpl<ExamineMapper, Examine> implements IExamineService {

    @Autowired
    ExamineMapper examineMapper;
    @Autowired
    ExamineRecordMapper recordMapper;
    @Autowired
    ExamineDetailMapper examineDetailMapper;
    @Autowired
    WxMpService wxMpService;
    @Autowired
    UserInfoMapper userInfoMapper;
    @Autowired
    IUserInfoService userInfoService;
    @Autowired
    BookingMapper bookingMapper;
    @Autowired
    BookingDetailMapper bookingDetailMapper;
    @Autowired
    ErrorOrderMapper errorOrderMapper;
    @Autowired
    OutOrderMapper outOrderMapper;
    @Autowired
    BoxOrderMapper boxOrderMapper;
    @Autowired
    WxUtil wxUtil;
    @Autowired
    IExamineDetailService examineDetailService;
    @Autowired
    PushBaseMapper pushBaseMapper;
    @Autowired
    IPushBaseService pushBaseService;
    @Autowired
    IOperateRecordService operateRecordService;
    @Autowired

```

```

    IUserSupplierService userSupplierService;
    @Autowired
    IDataDictionaryDetailService detailService;
    /**
     *
     * @return
     */
    @Override
    public List<Examine> findList() {
        QueryWrapper<Examine> queryWrapper = new QueryWrapper<>();
        queryWrapper.eq("IsDeleted", 0);
        List<Examine> examineList = examineMapper.selectList(queryWrapper);
        return examineList;
    }
    /**
     * ID
     * @param examineId
     * @return
     */
    @Override
    public List<ExamineDetail> getDetailByExamineId(String examineId) {
        QueryWrapper<ExamineDetail> queryWrapper = new QueryWrapper<ExamineDetail>()
            .eq("IsDeleted", 0)
            .eq("ExamineId", examineId)
            .orderByAsc("Sort");
        List<ExamineDetail> examineList = examineDetailMapper.selectList(queryWrapper);
        return examineList;
    }
    /**
     *
     * @param orderId
     * @return
     */
    @Override
    public List<ExamineDetail> getExamineDetailList(String orderId) {
        QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
            .eq("OrderId", orderId)
            .eq("Sort", 1);
        ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
    }
    /**
     * ID
     * @param examineName
     * @return
     */
    @Override
    public String getExamineIdByName(String examineName) {
        Examine examine = examineMapper.selectOne(new QueryWrapper<Examine>().eq("Title", examineName));
        return examine.getId();
    }
    /**

```

```

*
* ID
* @return
*/
@Override
public List<BookingDetail> findUrgentByUserId() {
    String userId = TokenUtil.getUserId();
    return examineMapper.findUrgentByUserId(userId);
}
/**
*
* @param orderId
* @param checkStatus
* @param message
*/
@Override
@Transactional
public void examineBoxOrder(String orderId, Integer checkStatus, String message) {
    1. 、
    QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("IsDeleted", 0)
        .eq("OrderId", orderId)
        .eq("CheckStatus", 1);
    ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
    examineRecord.setCheckStatus(checkStatus);
    examineRecord.setMessage(message);
    recordMapper.updateById(examineRecord);
    UserInfo storeKeeper = userInfoMapper.findStoreKeeperByOrderId("BoxOrder", orderId);
    BoxOrder boxOrder = boxOrderMapper.selectById(orderId);
    UserInfo driver = userInfoMapper.selectById(boxOrder.getBookingUserId());
    2.
    UpdateWrapper updateWrapper = null;
    if (checkStatus == 2) {
        String examineId = examineRecord.getExamineId();
        int sort = examineRecord.getSort() + 1;
        queryWrapper = new QueryWrapper<ExamineDetail>()
            .eq("ExamineId", examineId).eq("Sort", sort);
        1.
        ExamineDetail examineDetail = examineDetailMapper.selectOne(queryWrapper);
        if (examineDetail == null) {
            2
            updateWrapper = new UpdateWrapper<BoxOrder>()
                .eq("Id", orderId)
                .set("CheckStatus", "2");
            boxOrderMapper.update(new BoxOrder(), updateWrapper);

            Sys_Qjck_Sptg
            if ("1".equals(detailService.getValueByDictCode("Sys_Qjck_Sptg"))) {
                pushBaseService.pushSysMsg(null, storeKeeper.getId(), storeKeeper.getName(),
                    "", 1);
            }
        }
    }
}

```

```

        pushBaseService.pushSysMsg(null, driver.getId(), driver.getName(),
            "", 3);
    }

    if ("1".equals(detailService.getValueByDictCode("Order"))) {
        /*Map<String, String> dataMap = new HashMap<>();
        dataMap.put("thing9", "");
        dataMap.put("phrase5", "");
        dataMap.put("thing8", message);
        dataMap.put("date7", DateUtil.getDate("yyyy-MM-dd HH:mm:ss"));
        wxUtil.sendWxSubscribeMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order.getValue(), dataMap);
        wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);*/

        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        {{remark.DATA}}
        wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
        wxUtil.sendWxTemplateMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
    }

    return;
}

IDataDictionaryDetailService detailService;
/**
 *
 * @return
 */
@Override
public List<Examine> findList() {
    QueryWrapper<Examine> queryWrapper = new QueryWrapper<>();
    queryWrapper.eq("IsDeleted", 0);
    List<Examine> examineList = examineMapper.selectList(queryWrapper);
    return examineList;
}
/**
 * ID
 * @param examineId
 * @return
 */
@Override
public List<ExamineDetail> getDetailByExamineId(String examineId) {
    QueryWrapper<ExamineDetail> queryWrapper = new QueryWrapper<ExamineDetail>()

```

```

        .eq("IsDeleted", 0)
        .eq("ExamineId", examineId)
        .orderByAsc("Sort");

    List<ExamineDetail> examineList = examineDetailMapper.selectList(queryWrapper);
    return examineList;
}

/**
 *
 * @param orderId
 * @return
 */
@Override
public List<ExamineDetail> getExamineDetailList(String orderId) {
    QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("OrderId", orderId)
        .eq("Sort", 1);
    ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
    List<ExamineDetail> examineDetailList = examineMapper.getExamineDetailList(examineRecord.getExamineId());
    return examineDetailList;
}

/**
 *
 * @param examineDetailList
 */
@Override
@Transactional
public void saveDetail(List<ExamineDetail> examineDetailList) {
    1.
    QueryWrapper queryWrapper = new QueryWrapper<ExamineDetail>()
        .eq("IsDeleted", 0)
        .eq("ExamineId", examineDetailList.get(0).getExamineId())
        .orderByAsc("Sort");
    List<ExamineDetail> oldList = examineDetailMapper.selectList(queryWrapper);
    queryWrapper = new QueryWrapper<ExamineDetail>()
        .eq("IsDeleted", 0)
        .eq("ExamineId", examineDetailList.get(0).getExamineId());
    examineDetailMapper.delete(queryWrapper);
    2.
    examineDetailService.saveBatch(examineDetailList);
    operateRecordService.insertRecord("", "- ", " " + oldList.toString() + ", " +
    examineDetailList.toString());
}

}

2.2
/**Map<String, String> dataMap = new HashMap<>();
dataMap.put("car_number1", pushBase.getVehicleNo());
dataMap.put("thing2", pushBase.getPushMessage());
wxUtil.sendWxSubscribeMsg(userInfo.getWeChatOpenId(),
TemplateTypeEnum.ContactStoreKeeper.getValue(), dataMap);*/
List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();

```

```

        {{first.DATA}}
    : {{keyword1.DATA}}
    : {{keyword2.DATA}}
    {{remark.DATA}}
    wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
    wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", pushBase.getVehicleNo()));
    wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", pushBase.getPushMessage()));
    wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
    wxUtil.sendWxTemplateMsg(userInfo.getWeChatOpenId(),
TemplateTypeEnum.ContactStoreKeeper_Template.getValue(), wxMpTemplateDataList);
    }
    /**
     *
     * @param bookingId
     * @param isReplaceId
     */
    @Override
    public List<BookingDetail> storeKeeperScan(String bookingId, String isReplaceId) {
        String userId = StringUtils.isNotBlank(isReplaceId) ? isReplaceId : TokenUtil.getUserId();
        List<BookingDetail> bookingDetailList = bookingMapper.findDetailByStoreKeeper(bookingId, userId);
        if (bookingDetailList == null || bookingDetailList.size() == 0) {
            throw new ServiceException("");
        }
        List<String> detailIdList = new ArrayList<>();
        boolean updateFlag = false;
        for (BookingDetail bookingDetail : bookingDetailList) {
            /*if (!"4".equals(bookingDetail.getBookingStatus())) {
                throw new ServiceException("");
            }*/
            detailIdList.add(bookingDetail.getId());
            if (bookingDetail.getScanTime() == null) {
                bookingDetail.setScanTime(new Date());
                updateFlag = true;
            }
            QueryWrapper queryWrapper = null;
            InOrder inOrder = inOrderMapper.findInOrder(bookingDetail.getId());
            if (inOrder != null) {
                List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "IN");
                inOrder.setPaperList(paperList);
            }
            bookingDetail.setInOrder(inOrder);
            queryWrapper = new QueryWrapper<InOrderDetail>()
                .eq("IsDeleted", 0)
                .eq("BookingDetailId", bookingDetail.getId())
                .orderByAsc("Sort");
            List<InOrderDetail> inOrderDetailList = inOrderDetailMapper.selectList(queryWrapper);
            bookingDetail.setInOrderDetailList(inOrderDetailList);
            BoxOrder boxOrder = boxOrderMapper.findBoxOrder(bookingDetail.getId());
            if (boxOrder != null) {
                List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "BOX");
            }
        }
    }

```

```

        boxOrder.setPaperList(paperList);
    }
    bookingDetail.setBoxOrder(boxOrder);
    OutOrder outOrder = outOrderMapper.findOutOrder(bookingDetail.getId());
    if (outOrder != null) {
        List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "OUT");
        outOrder.setPaperList(paperList);
    }
    bookingDetail.setOutOrder(outOrder);
    ErrorOrder errorOrder = errorOrderMapper.findErrorOrder(bookingDetail.getId());
    if (errorOrder != null) {
        List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetail.getId(), "ERR");
        errorOrder.setPaperList(paperList);
        queryWrapper = new QueryWrapper<FileUpload>()
            .eq("IsDeleted", 0)
            .eq("FileRecordType", 3)
            .eq("BookingDetailId", bookingDetail.getId());
        List<FileUpload> fileUploadList = fileUploadMapper.selectList(queryWrapper);
        errorOrder.setFileList(fileUploadList);
    }
    bookingDetail.setErrorOrder(errorOrder);
}

if (updateFlag) {
    bookingDetailService.updateBatchById(bookingDetailList);
    /* 1: 2: 3: 4:
    UpdateWrapper<QueueUp> updateWrapper = new UpdateWrapper<QueueUp>()
        .eq("IsDeleted", 0)
        .in("BookingDetailId", detailIdList)
        .set("QueueStatus", 3);
    queueUpMapper.update(new QueueUp(), updateWrapper);*/
}

return bookingDetailList;
}

}

package com.xinzhou.svcs.mobile.emp.service.impl;
import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.core.conditions.update.UpdateWrapper;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.xinzhou.svcs.common.config.DataInitialize;
import com.xinzhou.svcs.common.ex.InsertException;
import com.xinzhou.svcs.common.ex.UpdateException;
import com.xinzhou.svcs.common.utils.OrderUtil;
import com.xinzhou.svcs.common.utils.TokenUtil;
import com.xinzhou.svcs.common.utils.WxUtil;
import com.xinzhou.svcs.mobile.driver.mapper.BookingDetailMapper;
import com.xinzhou.svcs.mobile.emp.entity.*;
import com.xinzhou.svcs.mobile.emp.mapper.*;
import com.xinzhou.svcs.mobile.emp.service.IErrorOrderDetailService;
import com.xinzhou.svcs.mobile.emp.service.IErrorOrderService;
import com.xinzhou.svcs.mobile.emp.service.IExamineService;

```

```

import com.xinzhou.svcs.mobile.emp.service.IOrderService;
import com.xinzhou.svcs.mobile.register.entity.UserInfo;
import com.xinzhou.svcs.mobile.register.mapper.UserInfoMapper;
import com.xinzhou.svcs.mobile.register.service.IUserInfoService;
import com.xinzhou.svcs.pc.entity.FileUpload;
import com.xinzhou.svcs.pc.enums.TemplateTypeEnum;
import com.xinzhou.svcs.pc.mapper.FileUploadMapper;
import com.xinzhou.svcs.pc.service.IDataDictionaryDetailService;
import com.xinzhou.svcs.pc.service.IFileUploadService;
import com.xinzhou.svcs.pc.service.IPushBaseService;
import me.chanjar.weixin.mp.bean.template.WxMpTemplateData;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
/**
 * <p>
 *
 * </p>
 *
 * @since 2021-03-09
 */
@Service
public class ErrorOrderServiceImpl extends ServiceImpl<ErrorOrderMapper, ErrorOrder> implements
IErrorOrderService {
    @Autowired
    ErrorOrderMapper mapper;
    @Autowired
    ErrorOrderDetailMapper detailMapper;
    @Autowired
    IErrorOrderDetailService detailService;
    @Autowired
    ExamineDetailMapper examineDetailMapper;
    @Autowired
    ExamineRecordMapper examineRecordMapper;
    @Autowired
    OrderMapper orderMapper;
    @Autowired
    FileUploadMapper fileUploadMapper;
    @Autowired
    UserInfoMapper userInfoMapper;
    @Autowired
    IUserInfoService userInfoService;
    @Autowired
    WxUtil wxUtil;
    @Autowired

```

```

IOrderService orderService;
@Autowired
BookingDetailMapper bookingDetailMapper;
@Autowired
IDataDictionaryDetailsService dictDetailsService;
@Autowired
IPushBaseService pushBaseService;
@Autowired
IExamineService examineService;
@Autowired
IFileUploadService fileUploadService;
/**
 *
 * @param errorOrderList
 */
@Override
@Transactional
public void examineBoxOrder(String orderId, Integer checkStatus, String message) {
    1. 、
    QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("IsDeleted", 0)
        .eq("OrderId", orderId)
        .eq("CheckStatus", 1);
    ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
    examineRecord.setCheckStatus(checkStatus);
    examineRecord.setMessage(message);
    recordMapper.updateById(examineRecord);
    UserInfo storeKeeper = userInfoMapper.findStoreKeeperByOrderId("BoxOrder", orderId);
    BoxOrder boxOrder = boxOrderMapper.selectById(orderId);
    UserInfo driver = userInfoMapper.selectById(boxOrder.getBookingUserId());
    2.
    UpdateWrapper updateWrapper = null;
    if (checkStatus == 2) {
        String examineId = examineRecord.getExamineId();
        int sort = examineRecord.getSort() + 1;
        queryWrapper = new QueryWrapper<ExamineDetail>()
            .eq("ExamineId", examineId).eq("Sort", sort);
        1.
        ExamineDetail examineDetail = examineDetailMapper.selectOne(queryWrapper);
        if (examineDetail == null) {
            2
            updateWrapper = new UpdateWrapper<BoxOrder>()
                .eq("Id", orderId)
                .set("CheckStatus", "2");
            boxOrderMapper.update(new BoxOrder(), updateWrapper);

            Sys_Qjck_Sptg
            if ("1".equals(detailService.getValueByDictCode("Sys_Qjck_Sptg"))) {
                pushBaseService.pushSysMsg(null, storeKeeper.getId(), storeKeeper.getName(),
                    "", 1);
            }
        }
    }
}

```

```

        pushBaseService.pushSysMsg(null, driver.getId(), driver.getName(),
            "", 3);
    }

    if ("1".equals(detailService.getValueByDictCode("Order"))) {
        /*Map<String, String> dataMap = new HashMap<>();
        dataMap.put("thing9", "");
        dataMap.put("phrase5", "");
        dataMap.put("thing8", message);
        dataMap.put("date7", DateUtil.getDate("yyyy-MM-dd HH:mm:ss"));
        wxUtil.sendWxSubscribeMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order.getValue(), dataMap);
        wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);*/

        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        {{remark.DATA}}
        wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
        wxUtil.sendWxTemplateMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
    }

    return;
}

IDataDictionaryDetailService detailService;
/**
 *
 * @return
 */
@Override
public List<Examine> findList() {
    QueryWrapper<Examine> queryWrapper = new QueryWrapper<>();
    queryWrapper.eq("IsDeleted", 0);
    List<Examine> examineList = examineMapper.selectList(queryWrapper);
    return examineList;
}
/**
 * ID
 * @param examineId
 * @return
 */
@Override
public List<ExamineDetail> getDetailByExamineId(String examineId) {
    QueryWrapper<ExamineDetail> queryWrapper = new QueryWrapper<ExamineDetail>()

```

```

        .eq("IsDeleted", 0)
        .eq("ExamineId", examineId)
        .orderByAsc("Sort");

    List<ExamineDetail> examineList = examineDetailMapper.selectList(queryWrapper);
    return examineList;
}

/**
 *
 * @param orderId
 * @return
 * @Override
 * @Transactional
 */
public void addErrorOrder(List<ErrorOrder> errorOrderList) {
    String examineId = DataInitialize.examineMap.get("");
    ExamineDetail examineDetail = examineDetailMapper.findByExamineId(examineId).get(0);
    for (ErrorOrder errorOrder : errorOrderList) {
        UpdateWrapper updateWrapper = new UpdateWrapper<ErrorOrderDetail>()
            .eq("IsDeleted", 0)
            .eq("BookingDetailId", errorOrder.getBookingDetailId())
            .set("IsDeleted", 1);
        detailMapper.update(new ErrorOrderDetail(), updateWrapper);
        updateWrapper = new UpdateWrapper<ErrorOrder>()
            .eq("IsDeleted", 0)
            .eq("BookingDetailId", errorOrder.getBookingDetailId())
            .set("IsDeleted", 1);
        mapper.update(new ErrorOrder(), updateWrapper);
        String orderNo = OrderUtil.genOrderNo("ERR", errorOrder.getSupplierNo());
        errorOrder.setOrderNo(orderNo);
        if (errorOrder.getExamineFlag() != null && errorOrder.getExamineFlag() == 1) {
            errorOrder.setCheckStatus(2);
        } else {
            errorOrder.setCheckStatus(1);
        }
    }
    1.
    int insert = mapper.insert(errorOrder);
    if (insert != 1) {
        throw new InsertException("");
    }
    2.
    List<ErrorOrderDetail> errorOrderDetailList = errorOrder.getErrorOrderDetailList();
    for (ErrorOrderDetail errorOrderDetail : errorOrderDetailList) {
        errorOrderDetail.setOrderId(errorOrder.getId());
        errorOrderDetail.setOrderNo(orderNo);
        errorOrderDetail.setBookingDetailId(errorOrder.getBookingDetailId());
        errorOrderDetail.setBookingNo(errorOrder.getBookingNo());
        errorOrderDetail.setSupplierId(errorOrder.getSupplierId());
        errorOrderDetail.setSupplierNo(errorOrder.getSupplierNo());
        errorOrderDetail.setSupplierName(errorOrder.getSupplierName());
    }
    boolean b = detailService.saveBatch(errorOrderDetailList);
}

```

```

        if (!b) {
            throw new InsertException("");
        }
    }
    3.
    if (errorOrder.getExamineFlag() != null && errorOrder.getExamineFlag() == 0) {
        ExamineRecord examineRecord = new ExamineRecord();
        examineRecord.setOrderId(errorOrder.getId());
        examineRecord.setOrderNo(orderNo);
        1., 2., 3., 4.
        examineRecord.setExamineType(3);
        examineRecord.setRoleCode(examineDetail.getRoleCode());
        examineRecord.setRoleName(examineDetail.getRoleName());
        examineRecord.setSort(1);
        examineRecord.setExamineId(examineDetail.getExamineId()); ID
        examineRecord.setExamineDetailId(examineDetail.getId()); ID
        examineRecord.setCheckStatus(1); 1. 2. 3.

        UserInfo examineUser = examineService.getNextExamineUser(examineRecord, examineDetail);
        examineRecord.setExamineUserId(examineUser.getId());
        examineRecord.setExamineUserName(examineUser.getName());
        int rows = examineRecordMapper.insert(examineRecord);
        if (rows != 1) {
            throw new InsertException("");
        }

        if ("1".equals(dictDetailService.getValueByDictCode("Sys_Yclj_Xj"))) {
            pushBaseService.pushSysMsg(null, examineUser.getId(), examineUser.getName(),
                "", 1);
        }

        if ("1".equals(dictDetailService.getValueByDictCode("OrderNext"))) {
            /*Map<String, String> dataMap = new HashMap<>();
            dataMap.put("thing2", "");
            dataMap.put("thing3", "");
            wxUtil.sendWxSubscribeMsg(examineUser.getWeChatOpenId(),
            TemplateTypeEnum.OrderNext.getValue(), dataMap);*/

            List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
            {{first.DATA}}
            : {{keyword1.DATA}}
            : {{keyword2.DATA}}
            : {{keyword3.DATA}} initiator
            : {{keyword4.DATA}} submitter
            : {{keyword5.DATA}}
            {{remark.DATA}}

            UserInfo storeKeeper = userInfoMapper.selectById(TokenUtil.getUserId());
            wxMpTemplateDataList.add(new WxMpTemplateData("first", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", errorOrder.getOrderNo()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword3", storeKeeper.getName()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword4", storeKeeper.getName()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword5", ""));

```

```

        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(examineUser.getWeChatOpenId(),
TemplateTypeEnum.OrderNext_Template.getValue(), wxMpTemplateDataList);
    }
}

/**
 *
 * @param bookingDetailId
 */
@Override
@Transactional
public void deleteErrorOrder(String bookingDetailId) {
    UpdateWrapper updateWrapper = new UpdateWrapper<ErrorOrderDetail>()
        .eq("IsDeleted", 0)
        .eq("BookingDetailId", bookingDetailId)
        .set("IsDeleted", 1);
    detailMapper.update(new ErrorOrderDetail(), updateWrapper);
    updateWrapper = new UpdateWrapper<ErrorOrder>()
        .eq("IsDeleted", 0)
        .eq("BookingDetailId", bookingDetailId)
        .set("IsDeleted", 1);
    mapper.update(new ErrorOrder(), updateWrapper);
}

/**
 *
 * @param errorOrderId
 */
@Override
public void confirmErrorOrder(String errorOrderId) {
    UpdateWrapper<ErrorOrder> updateWrapper =
        new UpdateWrapper<ErrorOrder>().eq("Id", errorOrderId).set("OrderStatus", 1);
    int update = mapper.update(new ErrorOrder(), updateWrapper);
    if (update != 1) throw new UpdateException(", ");
}

/**
 *
 * @param isReplaceId
 * @return
 */
@Override
public List<ErrorOrder> findErrorOrderList(String isReplaceId) {
    String userId = StringUtils.isBlank(isReplaceId) ? isReplaceId : TokenUtil.getUserId();
    List<ErrorOrder> errorOrderList = mapper.findErrorOrderList(userId);
    return errorOrderList;
}

/**
 * ID
 * @param bookingDetailId

```

```

    * @return
    */
@Override
public ErrorOrder findErrorOrder(String bookingDetailId) {
    ErrorOrder errorOrder = mapper.findErrorOrder(bookingDetailId);
    if (errorOrder != null) {
        List<FileUpload> paperList = fileUploadService.getPaperList(bookingDetailId, "ERR");
        errorOrder.setPaperList(paperList);
        QueryWrapper queryWrapper = new QueryWrapper<FileUpload>()
        2.
        UserInfo currentExamineUser = userInfoMapper.selectById(examineRecord.getExamineUserId());
        UserInfo nextExamineUser = this.getNextExamineUser(examineRecord, examineDetail);
        this.saveExamineRecord(examineRecord, sort, nextExamineUser);
        3.
        queryWrapper = new QueryWrapper<UserInfo>()
            .eq("Id", nextExamineUser.getId());
        UserInfo userInfo = userInfoMapper.selectOne(queryWrapper);
        if ("1".equals(detailService.getValueByDictCode("Sys_Qjck_Sp"))) {
            pushBaseService.pushSysMsg(null, userInfo.getId(), userInfo.getName(),
                "", 1);
        }
        if ("1".equals(detailService.getValueByDictCode("OrderNext"))) {

            /*Map<String, String> dataMap = new HashMap<>();
            dataMap.put("thing2", "");
            dataMap.put("thing3", "");
            wxUtil.sendWxSubscribeMsg(userInfo.getWeChatOpenId(), TemplateTypeEnum.OrderNext.getValue(),
dataMap);*/

            List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
            {{first.DATA}}
            : {{keyword1.DATA}}
            : {{keyword2.DATA}}
            : {{keyword3.DATA}} initiator
            : {{keyword4.DATA}} submitter
            : {{keyword5.DATA}}
            {{remark.DATA}}

            queryWrapper = new QueryWrapper<ExamineRecord>()
                .eq("IsDeleted", 0)
                .eq("OrderId", orderId)
                .eq("Sort", 1);
            ExamineRecord firstExamineRecord = recordMapper.selectOne(queryWrapper);
            UserInfo initiator = userInfoMapper.selectById(firstExamineRecord.getCreatorId());

            String submitterName = examineRecord.getExamineUserName();
            wxMpTemplateDataList.add(new WxMpTemplateData("first", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", boxOrder.getOrderNo()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword3", initiator.getName()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword4", submitterName));

```

```

        wxMpTemplateDataList.add(new WxMpTemplateData("keyword5", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(userInfo.getWeChatOpenId(),
TemplateTypeEnum.OrderNext_Template.getValue(), wxMpTemplateDataList);
    }
}

if (checkStatus == 3) {
    CheckStatus 3;
    updateWrapper = new UpdateWrapper<BoxOrder>()
        .eq("Id", orderId)
        .set("CheckStatus", "3");
    boxOrderMapper.update(new BoxOrder(), updateWrapper);
    if ("1".equals(detailService.getValueByDictCode("Sys_Qjck_Spjj"))) {
        pushBaseService.pushSysMsg(null, storeKeeper.getId(), storeKeeper.getName(),
            ", : " + message, 1);
        pushBaseService.pushSysMsg(null, driver.getId(), driver.getName(),
            ", : " + message, 3);
    }
    if ("1".equals(detailService.getValueByDictCode("Order"))) {
        /*Map<String, String> dataMap = new HashMap<>();
        {{thing9.DATA}}
        {{phrase5.DATA}}
        {{thing8.DATA}}
        {{date7.DATA}}
        {{thing11.DATA}}
        dataMap.put("thing9", "");
        dataMap.put("phrase5", "");
        dataMap.put("thing8", message);
        dataMap.put("date7", DateUtil.getDate("yyyy-MM-dd HH:mm:ss"));
        wxUtil.sendWxSubscribeMsg(storeKeeper.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);

        wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);*/

        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        {{remark.DATA}}
        wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", "", " + message));
        wxUtil.sendWxTemplateMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
        wxUtil.sendWxTemplateMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
    }
}
}
}

```

```

/**
 *
 * @param orderId      ID
 * @param checkStatus
 * @param message
 */
@Override
@Transactional
public void examineOutOrder(String orderId, Integer checkStatus, String message) {
    1. 、
    QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("IsDeleted", 0)
        .eq("OrderId", orderId)
        .eq("CheckStatus", 1);
    ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
    examineRecord.setCheckStatus(checkStatus);
    examineRecord.setMessage(message);
    recordMapper.updateById(examineRecord);
    UserInfo storeKeeper = userInfoMapper.findStoreKeeperByOrderId("OutOrder", orderId);
    OutOrder outOrder = outOrderMapper.selectById(orderId);
    UserInfo driver = userInfoMapper.selectById(outOrder.getBookingUserId());
    2.
    UpdateWrapper updateWrapper = null;
    if (checkStatus == 2) {
        String examineId = examineRecord.getExamineId();
        int sort = examineRecord.getSort() + 1;
        queryWrapper = new QueryWrapper<ExamineDetail>()
            .eq("ExamineId", examineId).eq("Sort", sort);
        1.
        ExamineDetail examineDetail = examineDetailMapper.selectOne(queryWrapper);
        if (examineDetail == null) {
            2
            updateWrapper = new UpdateWrapper<OutOrder>()
                .eq("Id", orderId)
                .set("CheckStatus", "2");
            outOrderMapper.update(new OutOrder(), updateWrapper);

            if ("1".equals(detailService.getValueByDictCode("Sys_Ccwl_Sptg"))) {
                pushBaseService.pushSysMsg(null, storeKeeper.getId(), storeKeeper.getName(),
                    "", 1);
                pushBaseService.pushSysMsg(null, driver.getId(), driver.getName(),
                    "", 3);
            }

            if ("1".equals(detailService.getValueByDictCode("Order"))) {
                /*Map<String, String> dataMap = new HashMap<>();
                dataMap.put("thing9", "");
                dataMap.put("phrase5", "");
                dataMap.put("thing8", message);

```

```

        dataMap.put("date7", DateUtil.getDate("yyyy-MM-dd HH:mm:ss"));
        wxUtil.sendWxSubscribeMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order.getValue(), dataMap);
        wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);*/

        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        {{remark.DATA}}
        wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
        wxUtil.sendWxTemplateMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
    }
    return;
}
2.
UserInfo nextExamineUser = this.getNextExamineUser(examineRecord, examineDetail);
this.saveExamineRecord(examineRecord, sort, nextExamineUser);
3.
queryWrapper = new QueryWrapper<UserInfo>()
    .eq("Id", nextExamineUser.getId());
UserInfo userInfo = userInfoMapper.selectOne(queryWrapper);
if ("1".equals(detailService.getValueByDictCode("Sys_Ccwl_Sp"))) {
    pushBaseService.pushSysMsg(null, userInfo.getId(), userInfo.getName(),
        "", 1);
}
if ("1".equals(detailService.getValueByDictCode("OrderNext"))) {
    /*Map<String, String> dataMap = new HashMap<>();
    dataMap.put("thing2", "");
    dataMap.put("thing3", "");
    wxUtil.sendWxSubscribeMsg(userInfo.getWeChatOpenId(), TemplateTypeEnum.OrderNext.getValue(),
dataMap);*/

    List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
    {{first.DATA}}
    : {{keyword1.DATA}}
    : {{keyword2.DATA}}
    : {{keyword3.DATA}} initiator
    : {{keyword4.DATA}} submitter
    : {{keyword5.DATA}}
    {{remark.DATA}}

    queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("IsDeleted", 0)
        .eq("OrderId", orderId)

```

```

        .eq("Sort", 1);
        ExamineRecord firstExamineRecord = recordMapper.selectOne(queryWrapper);
        UserInfo initiator = userInfoMapper.selectById(firstExamineRecord.getCreatorId());

        String submitterName = examineRecord.getExamineUserName();
        wxMpTemplateDataList.add(new WxMpTemplateData("first", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", outOrder.getOrderNo()));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword3", initiator.getName()));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword4", submitterName));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword5", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(userInfo.getWeChatOpenId(),
        TemplateTypeEnum.OrderNext_Template.getValue(), wxMpTemplateDataList);
    }
}

if (checkStatus == 3) {
    CheckStatus 3;
    updateWrapper = new UpdateWrapper<OutOrder>()
        .eq("Id", orderId)
        .set("CheckStatus", "3");
    outOrderMapper.update(new OutOrder(), updateWrapper);
    if ("1".equals(detailService.getValueByDictCode("Sys_Ccwl_Spjj"))) {
        pushBaseService.pushSysMsg(null, storeKeeper.getId(), storeKeeper.getName(),
            ", : " + message, 1);
        pushBaseService.pushSysMsg(null, driver.getId(), driver.getName(),
            ", : " + message, 3);
    }
    if ("1".equals(detailService.getValueByDictCode("Order"))) {
        /*Map<String, String> dataMap = new HashMap<>();
        dataMap.put("thing9", "");
        dataMap.put("phrase5", "");
        dataMap.put("thing8", message);
        dataMap.put("date7", DateUtil.getDate("yyyy-MM-dd HH:mm:ss"));
        wxUtil.sendWxSubscribeMsg(storeKeeper.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);

        wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);*/

        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        {{remark.DATA}}
        wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", "", " + message));
        wxUtil.sendWxTemplateMsg(storeKeeper.getWeChatOpenId(),
        TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
    }
}

```

```

    * @param message
    */
    @Override
    @Transactional
    public void examineErrorOrder(String orderId, Integer checkStatus, String message) {
        1. 、
        QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
            .eq("OrderId", orderId)
            .eq("CheckStatus", 1);
        ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
        examineRecord.setCheckStatus(checkStatus);
        examineRecord.setMessage(message);
        recordMapper.updateById(examineRecord);
        UserInfo storeKeeper = userInfoMapper.findStoreKeeperByOrderId("ErrorOrder", orderId);
        ErrorOrder errorOrder = errorOrderMapper.selectById(orderId);
        UserInfo driver = userInfoMapper.selectById(errorOrder.getBookingUserId());
        2.
        UpdateWrapper updateWrapper = null;
        if (checkStatus == 2) {
            String examineId = examineRecord.getExamineId();
            int sort = examineRecord.getSort() + 1;
            queryWrapper = new QueryWrapper<ExamineDetail>()
                .eq("ExamineId", examineId).eq("Sort", sort);
            1.
            ExamineDetail examineDetail = examineDetailMapper.selectOne(queryWrapper);
            if (examineDetail == null) {
                2
                updateWrapper = new UpdateWrapper<ErrorOrder>()
                    .eq("Id", orderId)
                    .set("CheckStatus", "2");
                errorOrderMapper.update(new ErrorOrder(), updateWrapper);

                if ("1".equals(detailService.getValueByDictCode("Sys_Yclj_Sptg"))) {
                    pushBaseService.pushSysMsg(null, storeKeeper.getId(), storeKeeper.getName(),
                        "", 1);
                    pushBaseService.pushSysMsg(null, driver.getId(), storeKeeper.getName(),
                        "", 3);
                }

                if ("1".equals(detailService.getValueByDictCode("Order"))) {
                    /*Map<String, String> dataMap = new HashMap<>();
                    dataMap.put("thing9", "");
                    dataMap.put("phrase5", "");
                    dataMap.put("thing8", message);
                    dataMap.put("date7", DateUtil.getDate("yyyy-MM-dd HH:mm:ss"));
                    wxUtil.sendWxSubscribeMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order.getValue(), dataMap);
                    wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);*/

```

```

        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        {{remark.DATA}}
        wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
        wxUtil.sendWxTemplateMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
    }
    return;
}
2.
UserInfo nextExamineUser = this.getNextExamineUser(examineRecord, examineDetail);
this.saveExamineRecord(examineRecord, sort, nextExamineUser);
3.
queryWrapper = new QueryWrapper<UserInfo>()
    .eq("Id", nextExamineUser.getId());
UserInfo userInfo = userInfoMapper.selectOne(queryWrapper);
if ("1".equals(detailService.getValueByDictCode("Sys_Yclj_Sp"))) {
    pushBaseService.pushSysMsg(null, userInfo.getId(), userInfo.getName(),
        "", 1);
}
if ("1".equals(detailService.getValueByDictCode("OrderNext"))) {
    /*Map<String, String> dataMap = new HashMap<>();
    dataMap.put("thing2", "");
    dataMap.put("thing3", "");
    wxUtil.sendWxSubscribeMsg(userInfo.getWeChatOpenId(), TemplateTypeEnum.OrderNext.getValue(),
dataMap);*/

    List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
    {{first.DATA}}
    : {{keyword1.DATA}}
    : {{keyword2.DATA}}
    : {{keyword3.DATA}} initiator
    : {{keyword4.DATA}} submitter
    : {{keyword5.DATA}}
    {{remark.DATA}}

    queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("IsDeleted", 0)
        .eq("OrderId", orderId)
        .eq("Sort", 1);
    ExamineRecord firstExamineRecord = recordMapper.selectOne(queryWrapper);
    UserInfo initiator = userInfoMapper.selectById(firstExamineRecord.getCreatorId());

    String submitterName = examineRecord.getExamineUserName();

```

```

        wxMpTemplateDataList.add(new WxMpTemplateData("first", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", errorOrder.getOrderNo()));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword3", initiator.getName()));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword4", submitterName));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword5", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
        wxUtil.sendWxTemplateMsg(userInfo.getWeChatOpenId(),
TemplateTypeEnum.OrderNext_Template.getValue(), wxMpTemplateDataList);
    }
}

if (checkStatus == 3) {
    CheckStatus 3;
    updateWrapper = new UpdateWrapper<ErrorOrder>()
        .eq("Id", orderId)
        .set("CheckStatus", "3");
    errorOrderMapper.update(new ErrorOrder(), updateWrapper);
    if ("1".equals(detailService.getValueByDictCode("Sys_Yclj_Spjj"))) {
        pushBaseService.pushSysMsg(null, storeKeeper.getId(), storeKeeper.getName(),
            ", : " + message, 1);
        pushBaseService.pushSysMsg(null, driver.getId(), driver.getName(),
            ", : " + message, 3);
    }
    if ("1".equals(detailService.getValueByDictCode("Order"))) {
        /*Map<String, String> dataMap = new HashMap<>();
        dataMap.put("thing9", "");
        dataMap.put("phrase5", "");
        dataMap.put("thing8", message);
        dataMap.put("date7", DateUtil.getDate("yyyy-MM-dd HH:mm:ss"));
        wxUtil.sendWxSubscribeMsg(storeKeeper.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);

        wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(), TemplateTypeEnum.Order.getValue(),
dataMap);*/

        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        {{remark.DATA}}
        wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", "", " + message));
        wxUtil.sendWxTemplateMsg(storeKeeper.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
        wxUtil.sendWxTemplateMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.Order_Template.getValue(), wxMpTemplateDataList);
    }
}
}
}

```

```

/**
 *
 * @param examineRecord
 * @param examineDetail
 * @return
 */
@Override
public UserInfo getNextExamineUser(ExamineRecord examineRecord, ExamineDetail examineDetail) {
    QueryWrapper queryWrapper;
    UserInfo currentExamineUser = userInfoMapper.selectById(examineRecord.getExamineUserId());
    String nextExamineRoleCode = examineDetail.getRoleCode();
    UserInfo examineUser = null;
    if (nextExamineRoleCode.equals("1")) {
        roleCode
        queryWrapper = new QueryWrapper<UserInfo>()
            .eq("RoleCode", "1")
            .eq("IsDeleted", 0);
        examineUser = userInfoMapper.selectOne(queryWrapper);
    } else if (nextExamineRoleCode.equals("5")) {

        String orderId = examineRecord.getOrderId();
        ExamineType (1., 2., 3., 4.)
        Integer examineType = examineRecord.getExamineType();
        switch (examineType) {

            case 1 :
                BookingDetail bookingDetail =
                    bookingDetailMapper.selectOne(new QueryWrapper<BookingDetail>().eq("BookingId",
orderId));
                ID
                examineUser
                =
userSupplierService.findStoreKeeperBySupplierId(bookingDetail.getSupplierId());
                break;

            case 2 :
                OutOrder outOrder = outOrderMapper.selectById(orderId);
                ID
                examineUser = userSupplierService.findStoreKeeperBySupplierId(outOrder.getSupplierId());
                break;

            case 3 :
                ErrorOrder errorOrder = errorOrderMapper.selectById(orderId);
                ID
                examineUser
                =
userSupplierService.findStoreKeeperBySupplierId(errorOrder.getSupplierId());
                break;

            case 4 :
                BoxOrder boxOrder = boxOrderMapper.selectById(orderId);
                ID

```

```

        examineUser = userSupplierService.findStoreKeeperBySupplierId(boxOrder.getSupplierId());
    }
    } else if (nextExamineRoleCode.equals("2")) {
        examineUser = userInfoService.findMonitor();
    } else if (nextExamineRoleCode.equals("3")) {

        String orderId = examineRecord.getOrderId();
        ExamineType (1., 2., 3., 4.)
        Integer examineType = examineRecord.getExamineType();
        switch (examineType) {

            case 1 :
                BookingDetail bookingDetail =
                    bookingDetailMapper.selectOne(new QueryWrapper<BookingDetail>().eq("BookingId",
orderId));
                ID
                examineUser
                =
userSupplierService.findPlannerBySupplierId(bookingDetail.getSupplierId());
                break;

            case 2 :
                OutOrder outOrder = outOrderMapper.selectById(orderId);
                ID
                examineUser = userSupplierService.findPlannerBySupplierId(outOrder.getSupplierId());
                break;

        }
    }
    return examineUser;
}
/**
 *
 * @param examineRecord
 * @param sort
 * @param examineDetail
 */
private void saveExamineRecord(ExamineRecord examineRecord, int sort, UserInfo examineUser) {
    ExamineRecord examineNew = new ExamineRecord();
    examineNew.setOrderId(examineRecord.getOrderId());
    examineNew.setOrderNo(examineRecord.getOrderNo());
    examineNew.setExamineType(examineRecord.getExamineType());
    examineNew.setExamineUserId(examineUser.getId());
    examineNew.setExamineUserName(examineUser.getName());
    examineNew.setRoleCode(examineUser.getRoleCode());
    examineNew.setRoleName(RolesEnum.getRoleName(Integer.valueOf(examineUser.getRoleCode())));
    examineNew.setSort(sort);
    examineNew.setExamineId(examineRecord.getExamineId());ID
    examineNew.setExamineDetailId(examineRecord.getId());ID
    examineNew.setCheckStatus(1);
    int insert = recordMapper.insert(examineNew);
    if (insert != 1) {

```

```

        throw new ServiceException();
    }
}
/**
 *
 * @param orderId ID
 * @param checkStatus () 1. 2. 3.
 * @param message
 * @return
 */
@Override
@Transactional
public void examineUrgent(String orderId, Integer checkStatus, String message) {
    1. 、
    QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("OrderId", orderId)
        .eq("CheckStatus", 1);
    ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
    examineRecord.setCheckStatus(checkStatus);
    examineRecord.setMessage(message);
    recordMapper.updateById(examineRecord);
    Booking booking = bookingMapper.findBookingById(orderId);
    String strDate = DateUtil.getStrDate(booking.getBookingTime(), null);
    BookingDetail bookingDetail = booking.getBookingDetailList().get(0);
    String beginTime = bookingDetail.getBookingBeginTime();
    String endTime = bookingDetail.getBookingEndTime();
    2.
    UpdateWrapper updateWrapper = null;
    if (checkStatus == 2) {
        String examineId = examineRecord.getExamineId();
        int sort = examineRecord.getSort() + 1;
        queryWrapper = new QueryWrapper<ExamineDetail>()
            .eq("ExamineId", examineId).eq("Sort", sort);

        ExamineDetail examineDetail = examineDetailMapper.selectOne(queryWrapper);
        if (examineDetail == null) {
            2. 2
            updateWrapper = new UpdateWrapper<BookingDetail>()
                .eq("BookingId", orderId)
                .set("BookingStatus", "2")
                .set("CheckStatus", "2");
            bookingDetailMapper.update(new BookingDetail(), updateWrapper);
            CheckStatus 3;
            updateWrapper = new UpdateWrapper<Booking>()
                .eq("Id", orderId)
                .set("CheckStatus", "2");
            bookingMapper.update(new Booking(), updateWrapper);

            UserInfo driver = userInfoMapper.findDriverByBookingId(orderId);

```

```

        if ("1".equals(detailService.getValueByDictCode("Sys_Jjyy_Sptg"))) {
            pushBaseService.pushSysMsg(null, driver.getId(), driver.getName(),
                "", "", 3);
        }
        {{phrase1.DATA}} {{car_number4.DATA}} {{character_string6.DATA}}
        {{thing2.DATA}} {{thing5.DATA}}

        if ("1".equals(detailService.getValueByDictCode("UrgentBooking"))) {
            /*Map<String, String> dataMap = new HashMap<>();
            dataMap.put("phrase1", "");
            dataMap.put("car_number4", booking.getVehicleNo());
            dataMap.put("character_string6", strDate + " " + beginTime + "~" + endTime);
            dataMap.put("thing2", bookingDetail.getQuayName());
            dataMap.put("thing5", "");
            wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.UrgentBooking.getValue(), dataMap);*/
            List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
            {{first.DATA}}
            : {{keyword1.DATA}}
            : {{keyword2.DATA}}
            {{remark.DATA}}
            wxMpTemplateDataList.add(new WxMpTemplateData("first", ""));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", booking.getBookingNo()));
            wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", bookingDetail.getQuayName()));
            wxMpTemplateDataList.add(new WxMpTemplateData("remark", booking.getVehicleNo() + " ", ""));
            wxUtil.sendWxTemplateMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.UrgentBooking_Template.getValue(), wxMpTemplateDataList);
        }

        return;
    }

    UserInfo nextExamineUser = this.getNextExamineUser(examineRecord, examineDetail);
    this.saveExamineRecord(examineRecord, sort, nextExamineUser);
3.1
    queryWrapper = new QueryWrapper<UserInfo>()
        .eq("Id", nextExamineUser.getId());
    UserInfo emp = userInfoMapper.selectOne(queryWrapper);
    if ("1".equals(detailService.getValueByDictCode("Sys_Jjyy_Sp"))) {
        pushBaseService.pushSysMsg(null, emp.getId(), emp.getName(),
            "", "", 1);
    }

    if ("1".equals(detailService.getValueByDictCode("UrgentBookingNext"))) {
        /*Map<String, String> dataMap = new HashMap<>();
        dataMap.put("character_string1", booking.getBookingNo());
        dataMap.put("time3", strDate);
        dataMap.put("phrase4", "");
        dataMap.put("thing5", "", "");
        wxUtil.sendWxSubscribeMsg(emp.getWeChatOpenId(),
TemplateTypeEnum.UrgentBookingNext.getValue(), dataMap);*/
        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();

```

```

        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        : {{keyword3.DATA}}
        : {{keyword4.DATA}}
        {{remark.DATA}}
wxMpTemplateDataList.add(new WxMpTemplateData("first", "", ""));
wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", booking.getBookingUserName()));
wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", booking.getVehicleNo()));
String keyword3 = DateUtil.getStrDate(bookingDetail.getBookingTime(), "yyyy-MM-dd")
        + "" + bookingDetail.getBookingBeginTime() + "~" + bookingDetail.getBookingEndTime();
wxMpTemplateDataList.add(new WxMpTemplateData("keyword3", keyword3));
wxMpTemplateDataList.add(new WxMpTemplateData("keyword4", booking.getBookingNo()));
wxMpTemplateDataList.add(new WxMpTemplateData("remark", ""));
wxUtil.sendWxTemplateMsg(emp.getWeChatOpenId(),
TemplateTypeEnum.UrgentBookingNext_Template.getValue(), wxMpTemplateDataList);
    }
}

if (checkStatus == 3) {
    BookingStatus 2.1    CheckStatus 3;
    updateWrapper = new UpdateWrapper<BookingDetail>()
        .eq("BookingId", orderId)
        .set("BookingStatus", "2.1")
        .set("CheckStatus", "3");
    bookingDetailMapper.update(new BookingDetail(), updateWrapper);
    CheckStatus 3;
    updateWrapper = new UpdateWrapper<Booking>()
        .eq("Id", orderId)
        .set("CheckStatus", "3");
    bookingMapper.update(new Booking(), updateWrapper);
    UserInfo driver = userInfoMapper.findDriverByBookingId(orderId);
    if ("1".equals(detailService.getValueByDictCode("Sys_Jjyy_Spjj"))) {
        pushBaseService.pushSysMsg(null, driver.getId(), driver.getName(),
            "", "", 3);
    }
    if ("1".equals(detailService.getValueByDictCode("UrgentBooking"))) {
        /*Map<String, String> dataMap = new HashMap<>();
        dataMap.put("phrase1", "");
        dataMap.put("car_number4", booking.getVehicleNo());
        dataMap.put("character_string6", strDate + "" + beginTime + "~" + endTime);
        dataMap.put("thing2", bookingDetail.getQuayName());
        dataMap.put("thing5", "");
        wxUtil.sendWxSubscribeMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.UrgentBooking.getValue(), dataMap);*/
        List<WxMpTemplateData> wxMpTemplateDataList = new ArrayList<>();
        {{first.DATA}}
        : {{keyword1.DATA}}
        : {{keyword2.DATA}}
        {{remark.DATA}}

```

```

        wxMpTemplateDataList.add(new WxMpTemplateData("first", ""));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword1", booking.getBookingNo()));
        wxMpTemplateDataList.add(new WxMpTemplateData("keyword2", bookingDetail.getQuayName()));
        wxMpTemplateDataList.add(new WxMpTemplateData("remark", booking.getVehicleNo() + " , " +
message));
        wxUtil.sendWxTemplateMsg(driver.getWeChatOpenId(),
TemplateTypeEnum.UrgentBooking_Template.getValue(), wxMpTemplateDataList);
    }
}
}
/**
 *
 * @param orderId
 * @return
 */
@Override
public ExamineRecord findSuperiorExamineRecord(String orderId) {
    QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("OrderId", orderId)
        .eq("CheckStatus", 1);
    ExamineRecord examineRecord = recordMapper.selectOne(queryWrapper);
    queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("OrderId", orderId)
        .eq("Sort", examineRecord.getSort() - 1);
    ExamineRecord superior = recordMapper.selectOne(queryWrapper);
    return superior;
}
/**
 *
 * @param orderId
 * @return
 */
@Override
public List<ExamineRecord> getExamineList(String orderId) {
    QueryWrapper queryWrapper = new QueryWrapper<ExamineRecord>()
        .eq("OrderId", orderId)
        .orderByAsc("Sort");
    List list = recordMapper.selectList(queryWrapper);
    return list;
}
}

package com.xinzhou.svcs.mobile.emp.service.impl;
import com.xinzhou.svcs.common.utils.DateUtil;
import com.xinzhou.svcs.common.utils.UploadUtil;
import com.xinzhou.svcs.mobile.emp.service.IFileService;
import com.xinzhou.svcs.mobile.emp.vo.OrderVO;
import com.xinzhou.svcs.pc.entity.FileUpload;
import com.xinzhou.svcs.pc.enums.FileRecordTypeEnum;
import com.xinzhou.svcs.pc.mapper.FileUploadMapper;
import com.xinzhou.svcs.pc.service.IDataDictionaryDetailService;

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.ArrayList;
import java.util.List;
/**
 * <p>
 *
 * </p>
 *
 * @since 2021-03-01
 */
@Service
public class FileServiceImpl implements IFileService {
    @Autowired
    UploadUtil uploadUtil;
    @Autowired
    IDataDictionaryDetailService dictionaryDetailService;
    @Autowired
    FileUploadMapper fileUploadMapper;
    /**
     *
     * @param orderVO
     */
    @Override
    public List<FileUpload> uploadFile(OrderVO orderVO) {
        List<FileUpload> list = new ArrayList<>();
        /( )
        String path = DateUtil.getDate("yyyyMMdd") + "/" + orderVO.getBookingNo() + "/" +
FileRecordTypeEnum.getValue(orderVO.getFileRecordType());
        if (orderVO.getFileRecordType() == FileRecordTypeEnum.ERROR.getKey()) {
            path += "/" + orderVO.getSupplierName();
        }
        fileUpload
        if (orderVO.getFileList() != null && orderVO.getFileList().size() > 0) {
            list = uploadUtil.uploadFile(orderVO, path);
        }
        return list;
    }
    /**
     *
     * @param fileId
     */
    @Override
    public void delFile(String fileId) {
        FileUpload fileUpload = fileUploadMapper.selectById(fileId);
        fileUpload.setIsDeleted(1);
        fileUploadMapper.updateById(fileUpload);
    }
}

```